Laboratorio 4: REST APIs

HTTP, consumo desde Java, protocolos y diseño de una API con OpenAPI

Ingeniería de Software II (2016702)

Daniel Alonso Gracia Pinto

Noviembre de 2026

Grupo 00

Ejercicio 0. Anatomía de una petición HTTP

Las peticiones van a `https://httpbin.org`, que devuelve en el cuerpo lo que recibió: los parámetros, los headers y el cuerpo de la petición.

(a) Partes de la URL

La petición es `GET https://httpbin.org/get?curso=seii&grupo=3`.

Parte	Valor
Esquema	`https`
Host	`httpbin.org`
Ruta	`/get`
Parámetros de consulta	`curso=seii`, `grupo=3`

El signo `?` separa la ruta de los parámetros y `&` separa un parámetro del siguiente. En la respuesta, los parámetros aparecen en el campo `args`, ya convertidos en un objeto: `{"curso": "seii", "grupo": "3"}`. El `3` llega como texto, porque en la URL todo es texto y el servidor no sabe que era un número.

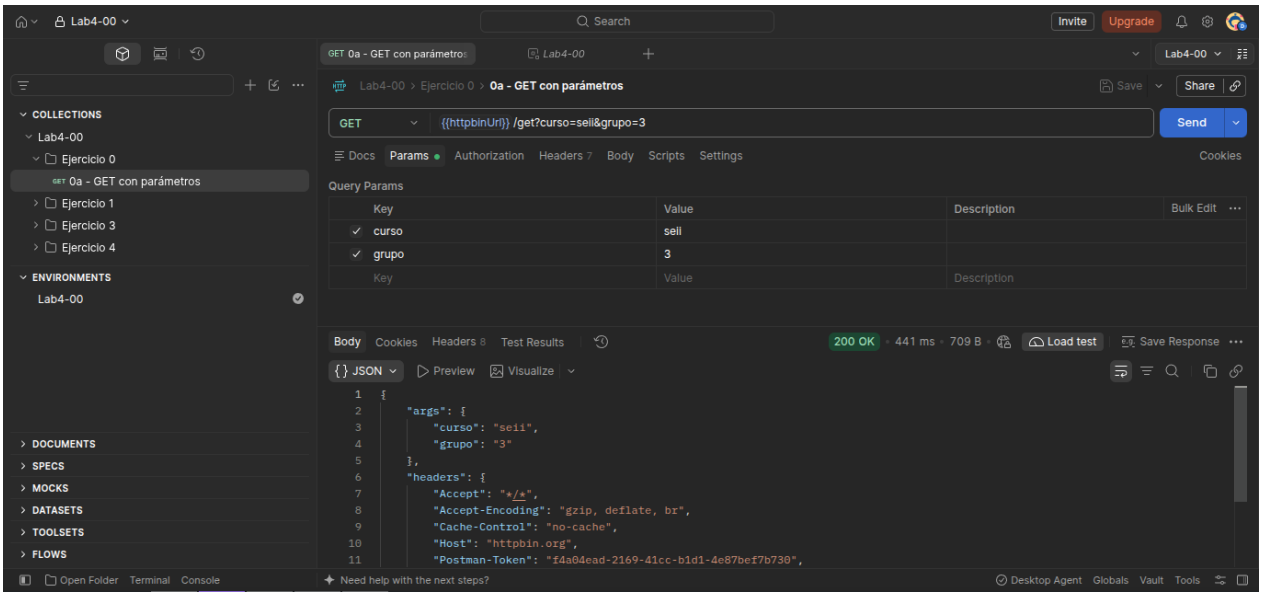


Figura 1. `GET /get` con los parámetros `curso` y `grupo`.

(b) Headers

Además del `X-Equipo: Musubi Japan`, Postman agregó por su cuenta siete headers: `Cache-Control` (`no-cache`), `Postman-Token` (un identificador distinto en cada envío), `Host`, `User-Agent` (`PostmanRuntime/2.12.2`), `Accept` (`*/*`), `Accept-Encoding` (`gzip, deflate, br`) y `Connection` (`keep-alive`). El campo `headers` de la respuesta los muestra todos menos `Connection`: ese header solo vale entre dos equipos vecinos de la red, y el intermediario que tiene httpbin.org adelante no lo reenvía.

`User-Agent`. Identifica el programa que hace la petición. El servidor lo usa para estadísticas, para adaptar la respuesta o para bloquear clientes. Con `curl` la misma petición llega con `curl/8.18.0`.

`Accept`. Dice qué formatos de respuesta acepta el cliente. `*/*` significa cualquiera; una app que solo entiende JSON debería enviar `application/json`.

`Host`. Dice a qué sitio va la petición. Un mismo servidor, con una sola IP, puede atender muchos dominios, y por el `Host` sabe cuál se pidió. En HTTP/1.1 es obligatorio.

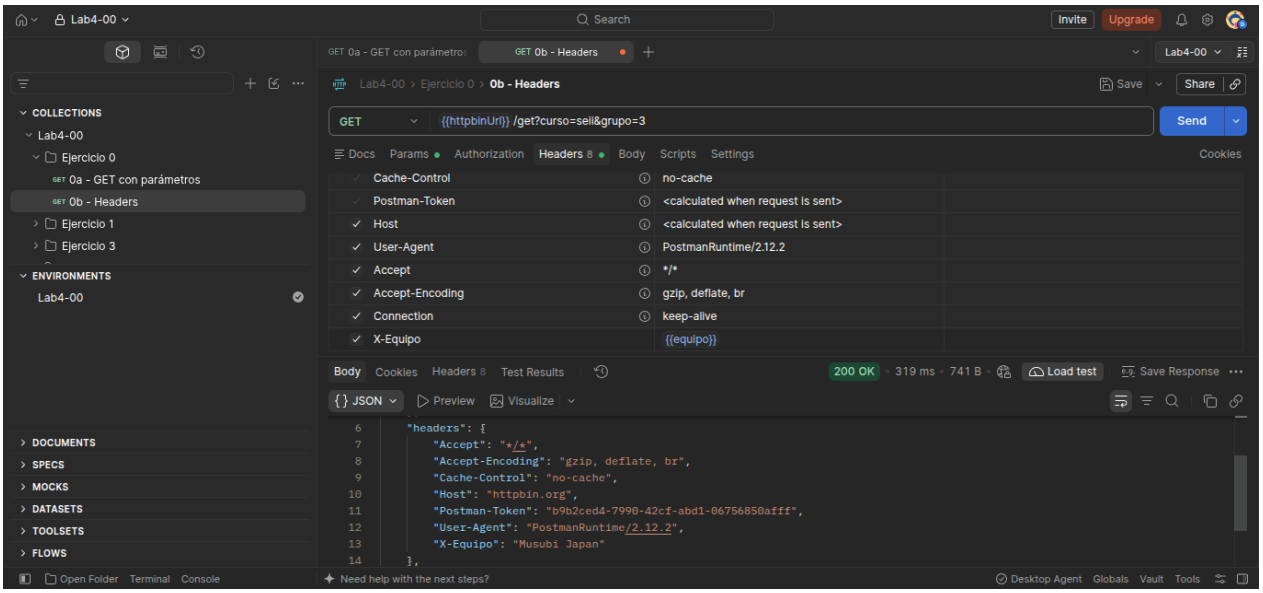


Figura 2. La misma petición con el header `X-Equipo`.

(c) Cuerpo JSON

El cuerpo enviado a `POST https://httpbin.org/post` es:

```
{
  "equipo": "Musubi Japan",
  "grupo": 0,
  "integrantes": ["Daniel Alonso Gracia Pinto"],
  "proyecto": "App móvil para aprender japonés"
}
```

El header que le dice al servidor que el cuerpo es JSON es `Content-Type: application/json`. Postman lo pone solo al elegir *raw* y *JSON*. La respuesta trae el cuerpo dos veces: en `data`, como texto, y en `json`, ya convertido en objeto.

Al cambiar a *Text*, Postman envía `Content-Type: text/plain`. El httpbin.org de hoy sigue llenando `json` con ese header: intenta leer el cuerpo como JSON sin mirar el `Content-Type`. Lo comprobé con un cuerpo que no es JSON (`hola equipo`): ahí `json` sale en `null` y el texto solo queda en `data`. El httpbin original, en cambio, devolvía `json: null` siempre que el tipo no fuera `application/json`. Eso es lo que debería hacer una API real: no adivinar el formato, sino responder `415 Unsupported Media Type` si el `Content-Type` no es el que espera.

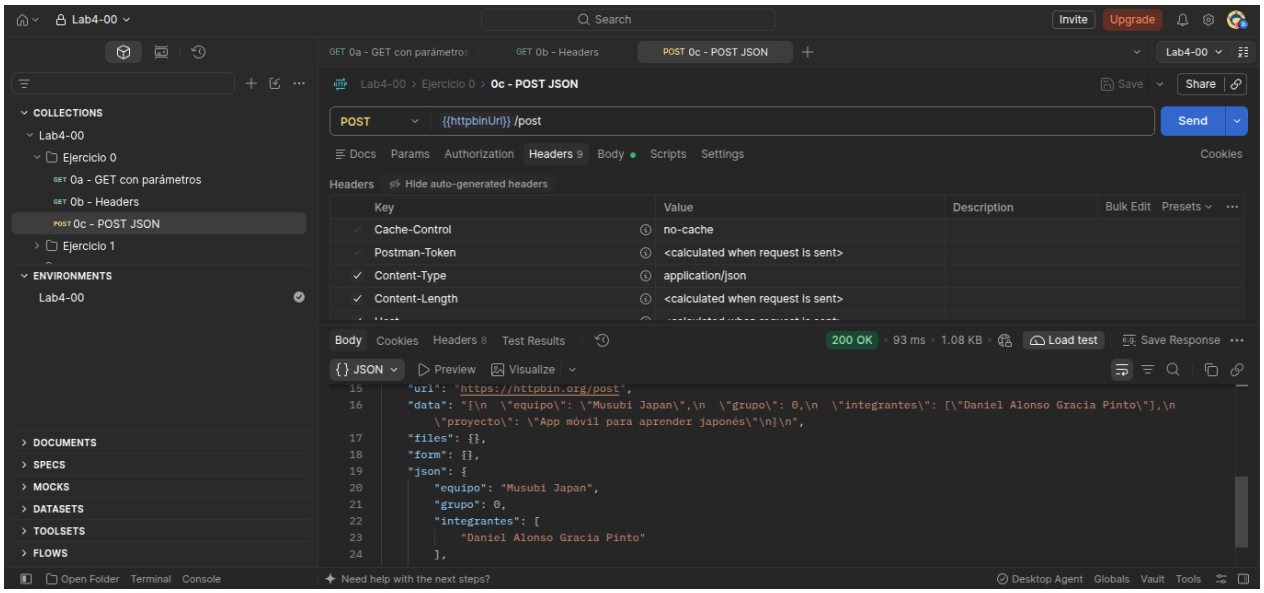


Figura 3. `POST /post` con el cuerpo como JSON.

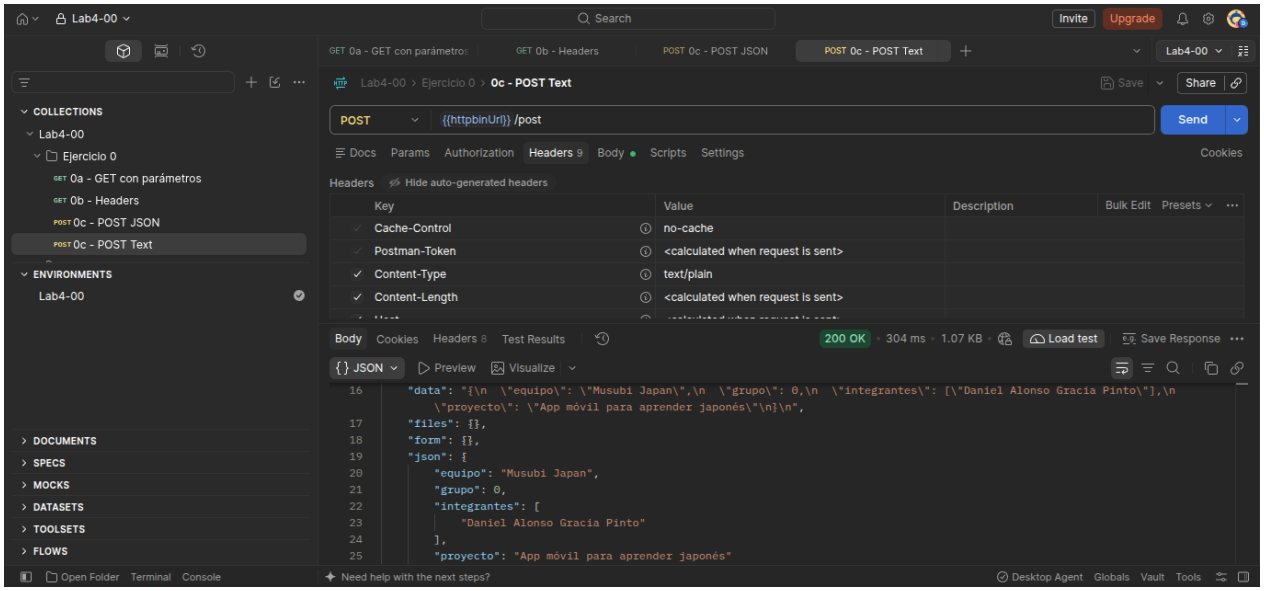


Figura 4. El mismo cuerpo enviado como *Text*: `json` sigue lleno.

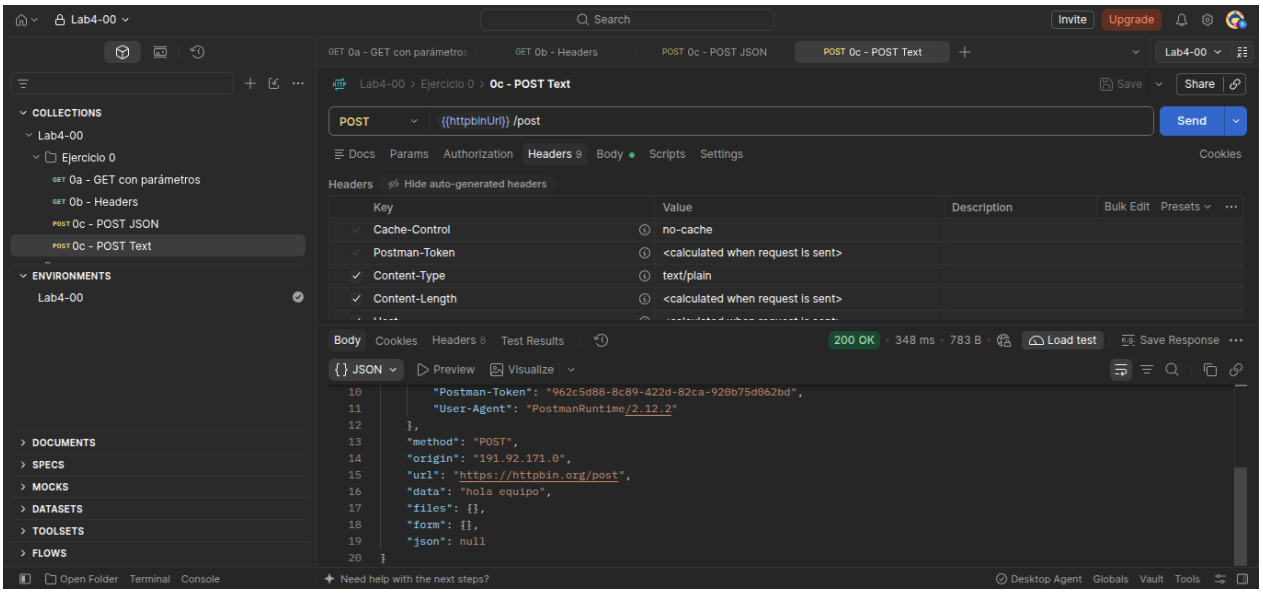


Figura 5. Un cuerpo *Text* que no es JSON: `json` sale en `null`.

(d) Códigos de estado

Las tres respuestas llegan con el cuerpo vacío (`Content-Length: 0`). La app solo cuenta con el código para decidir qué mostrar.

Código	Familia	Responsable	Qué muestra la app
201 Created	2xx	No hay error: el servidor creó el recurso.	Una confirmación (“Préstamo registrado”) y el recurso nuevo.
404 Not Found	4xx	El cliente: pidió un recurso que no existe o una URL mal armada.	“No encontramos lo que buscas” y la opción de volver. Reintentar no sirve.
500 Internal Server Error	5xx	El servidor: falló al procesar una petición válida.	“Algo salió mal de nuestro lado, intenta más tarde”, con un botón para reintentar y sin detalles técnicos.
429 Too Many Requests	4xx	El cliente: superó el límite de peticiones del servidor.	Nada de inmediato: espera lo que indique el servidor y reintenta sola. Si tarda, “Hay mucho tráfico, reintentando…”.

El 429 no se pidió: apareció al enviar la petición de (a). httpbin.org admite 17 peticiones por segundo por IP. Postman estaba usando su *Cloud Agent*, que envía la petición desde los servidores de Postman, con una IP que comparten muchos usuarios a la vez; la misma petición enviada con `curl` desde el equipo respondió 200. Al pasar al agente de escritorio, la petición salió con la IP propia y el 429 desapareció. A diferencia de los códigos de `/status`, este trae cuerpo, con el motivo y el tiempo de espera: `retry after 1 second`. Reintentar sí sirve aquí, pero esperando; reintentar enseguida solo alarga el bloqueo.

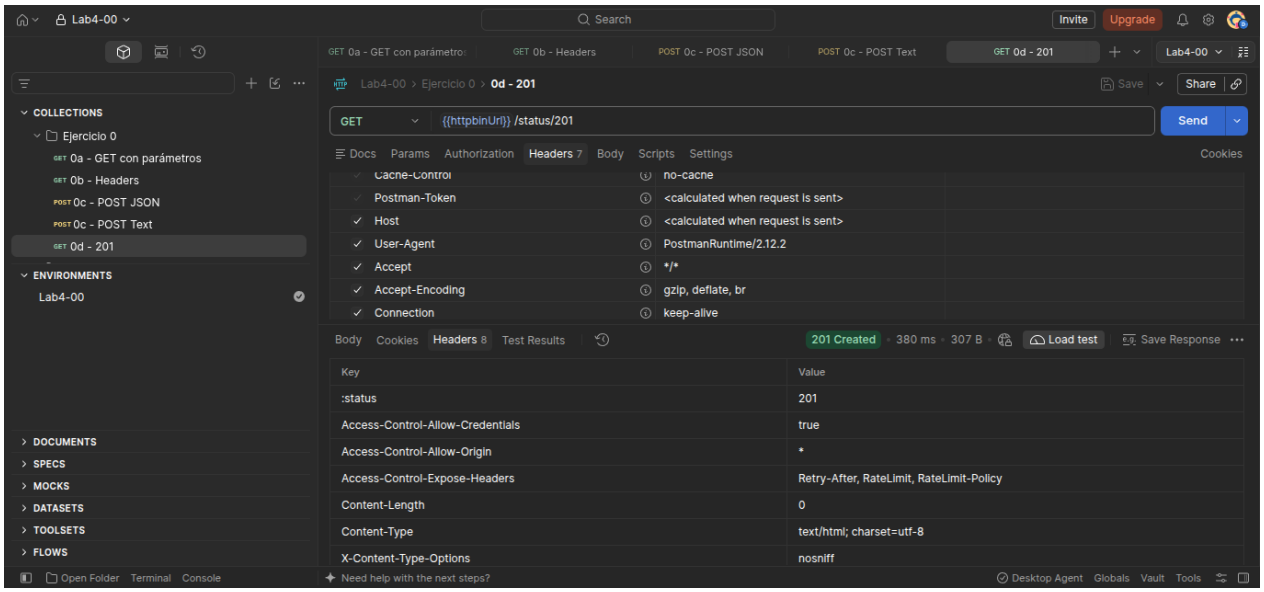


Figura 6. Respuesta de `/status/201`.

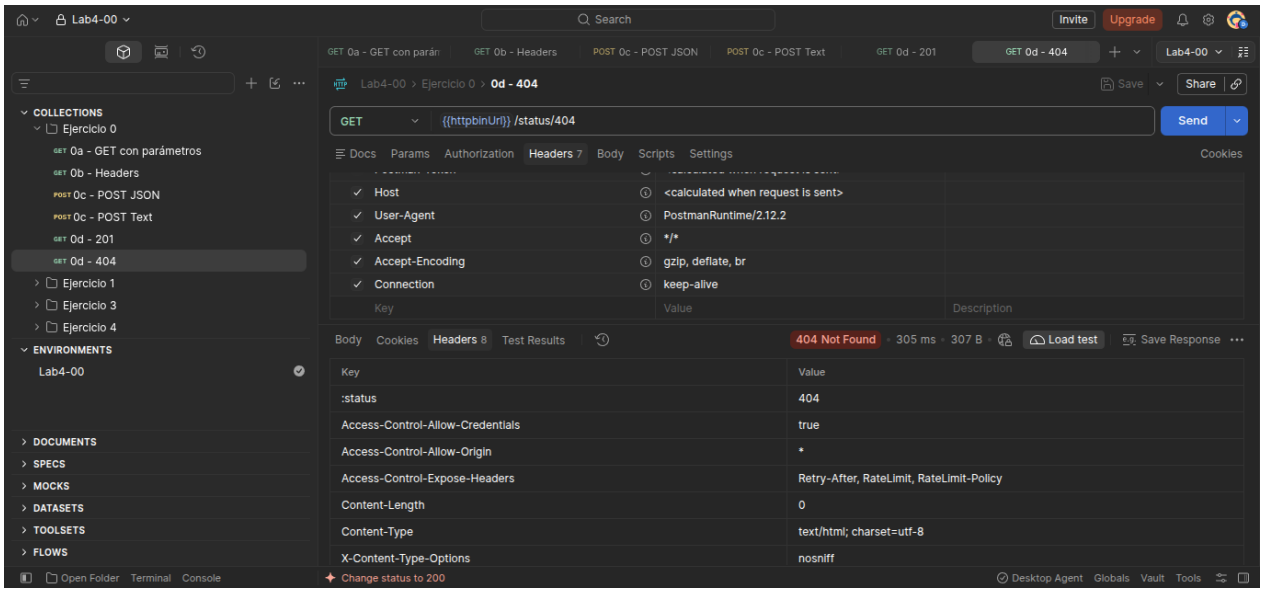


Figura 7. Respuesta de `/status/404`.

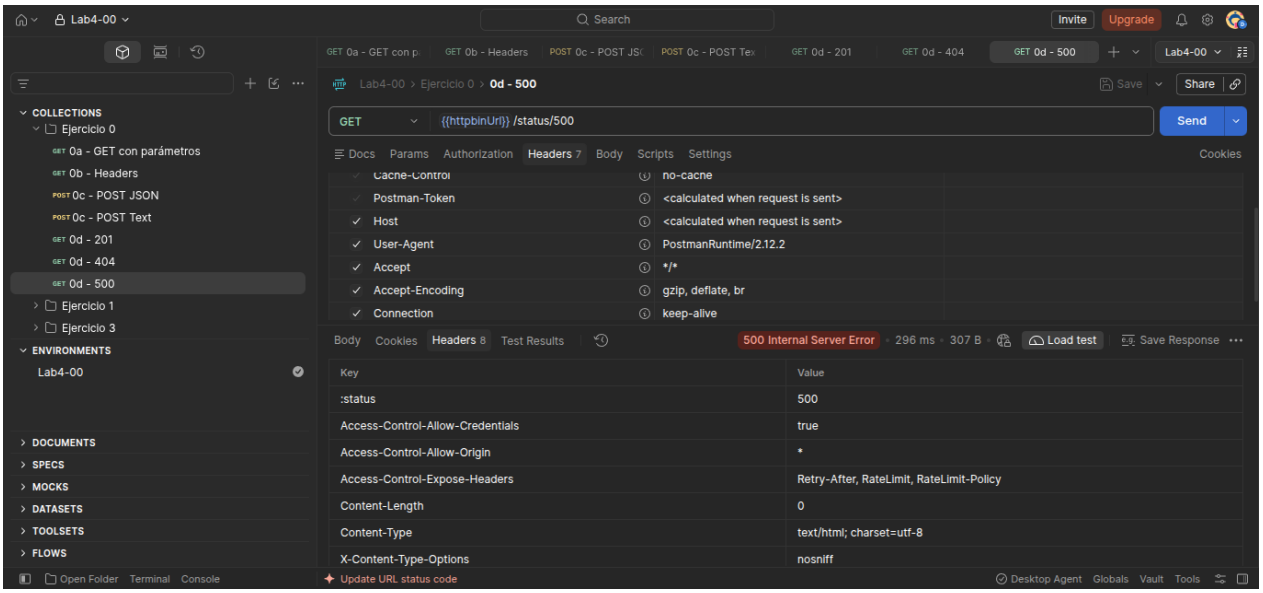


Figura 8. Respuesta de `/status/500`.

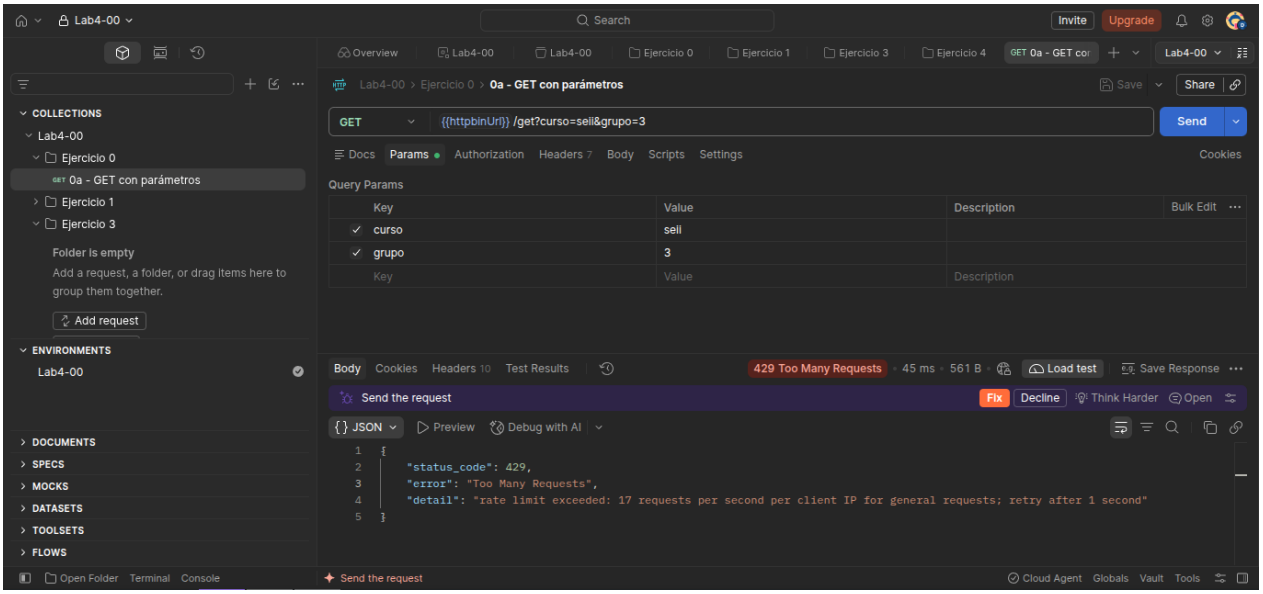


Figura 9. El 429 que devolvió httpbin.org al superar el límite de peticiones.

Ejercicio 1. CRUD completo con JSONPlaceholder

El environment define `baseUrl = https://jsonplaceholder.typicode.com`, `postId = 1` y `userId = 1`. Las peticiones usan esas variables en las rutas y en los cuerpos; la tabla muestra las URLs ya resueltas. JSONPlaceholder acepta escrituras, pero no las guarda: responde lo que respondería un servidor real.

(a) Métodos, códigos y propiedades

Un método es *seguro* si no cambia nada en el servidor, e *idempotente* si enviarlo una vez o varias deja el servidor en el mismo estado. Todo método seguro es idempotente; al revés no.

Método	Endpoint	Código	¿Seguro?	¿Idempotente?	Qué hace
GET	`/users`	200	Sí	Sí	Lista los 10 usuarios.
GET	`/users/1/posts`	200	Sí	Sí	Lista los 10 posts del usuario 1.
GET	`/posts?userId=1`	200	Sí	Sí	Filtra los posts por `userId`; devuelve los mismos 10.
POST	`/posts`	201	No	No	Crea un post. Responde con `id: 101` y `Location: /posts/101`.
PUT	`/posts/1`	200	No	Sí	Reemplaza el post 1 entero por el cuerpo enviado.
PATCH	`/posts/1`	200	No	No*	Cambia solo los campos enviados y conserva los demás.
DELETE	`/posts/1`	200	No	Sí	Borra el post 1. Responde `{}`.
GET	`/posts/9999`	404	Sí	Sí	Busca un post que no existe. Responde `{}`.

*HTTP no garantiza que `PATCH` sea idempotente. Lo es cuando fija valores, como aquí (`title = "Solo cambia el titulo"`), y deja de serlo con operaciones relativas, como "sumar 1 a las vistas".

`DELETE` es idempotente aunque la segunda vez responda distinto (en una API real, 404): lo que cuenta es el estado del servidor, y después de uno o de diez `DELETE` el post sigue sin existir. Una API real respondería `204 No Content` en lugar de `200` con un cuerpo vacío.

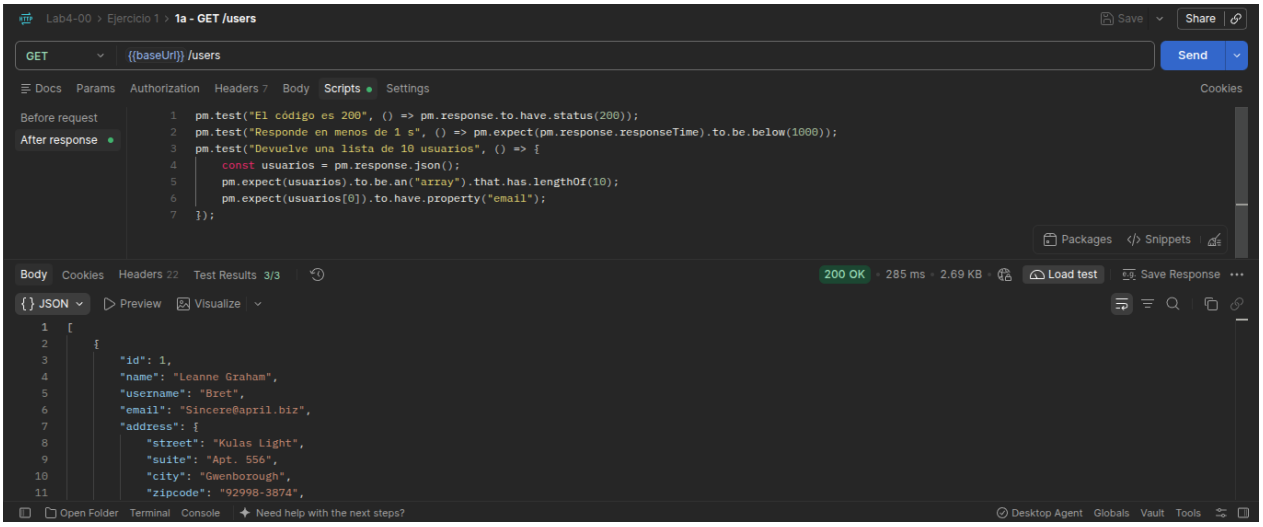


Figura 10. `GET /users`.

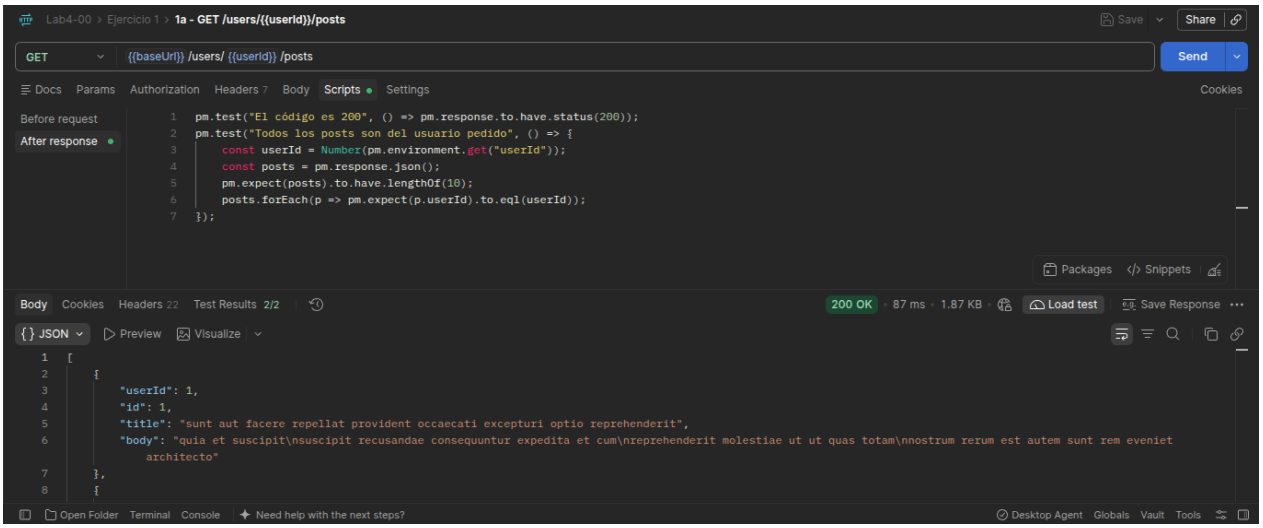


Figura 11. `GET /users/1/posts`.

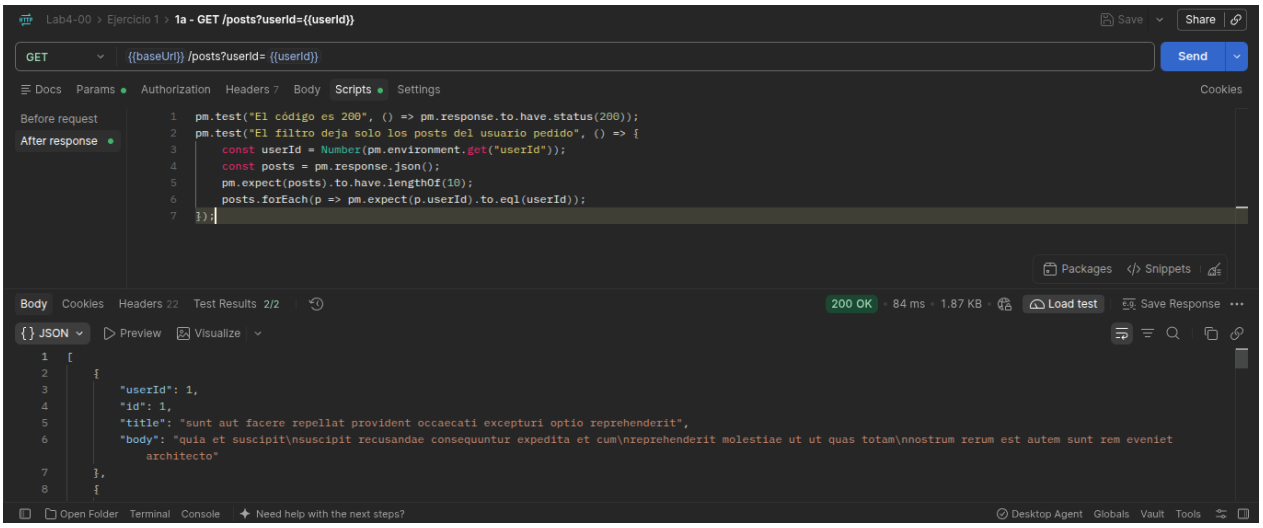


Figura 12. `GET /posts?userId=1`.

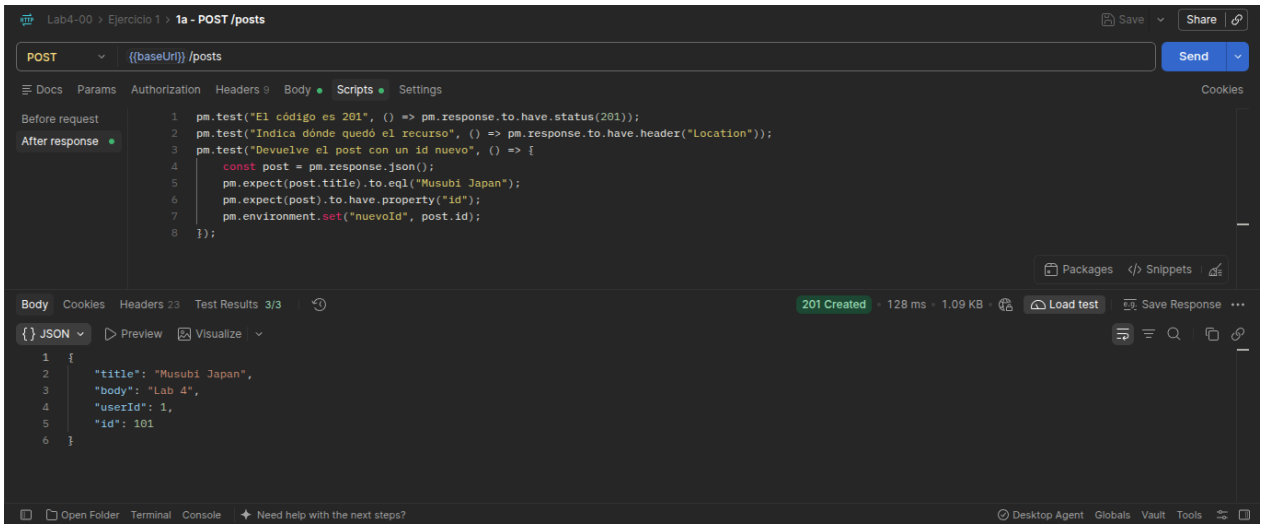


Figura 13. `POST /posts`: el cuerpo de la respuesta.

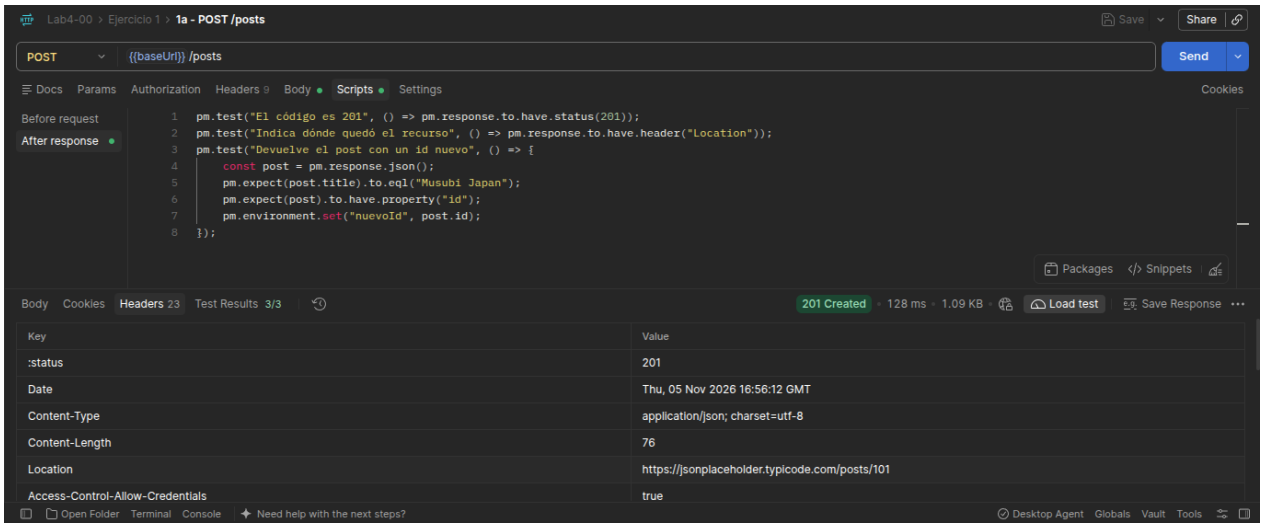


Figura 14. `POST /posts`: el header `Location` de la respuesta.

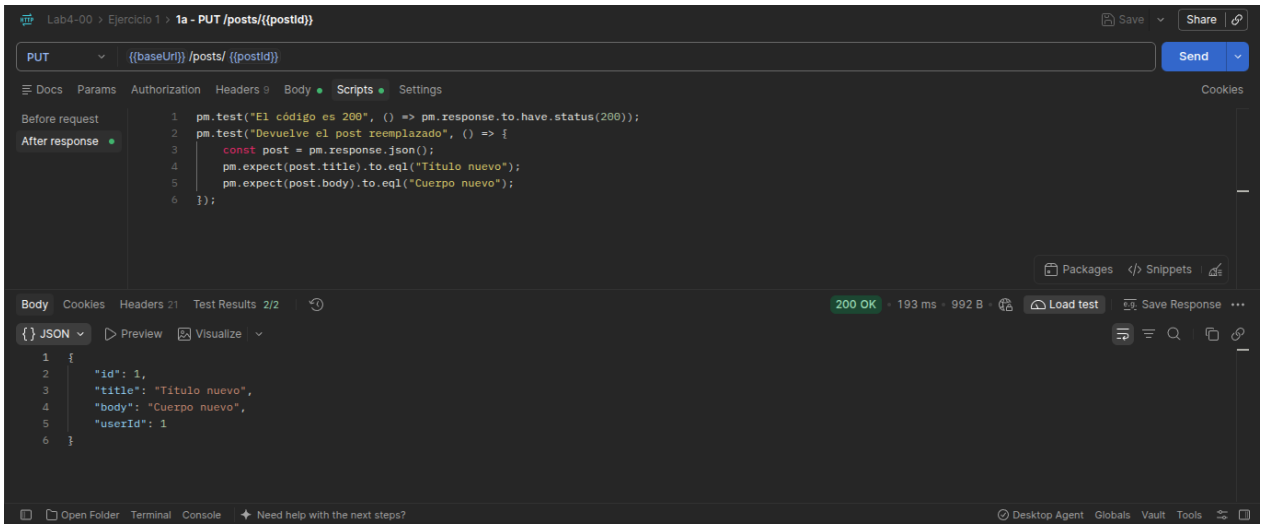


Figura 15. `PUT /posts/1`.

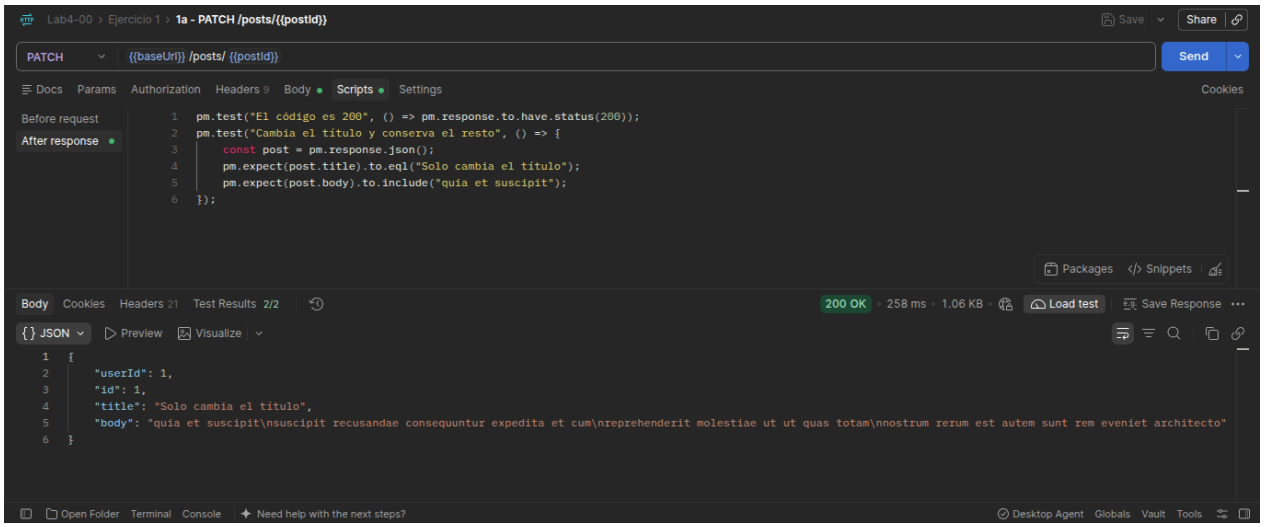


Figura 16. `PATCH /posts/1`.

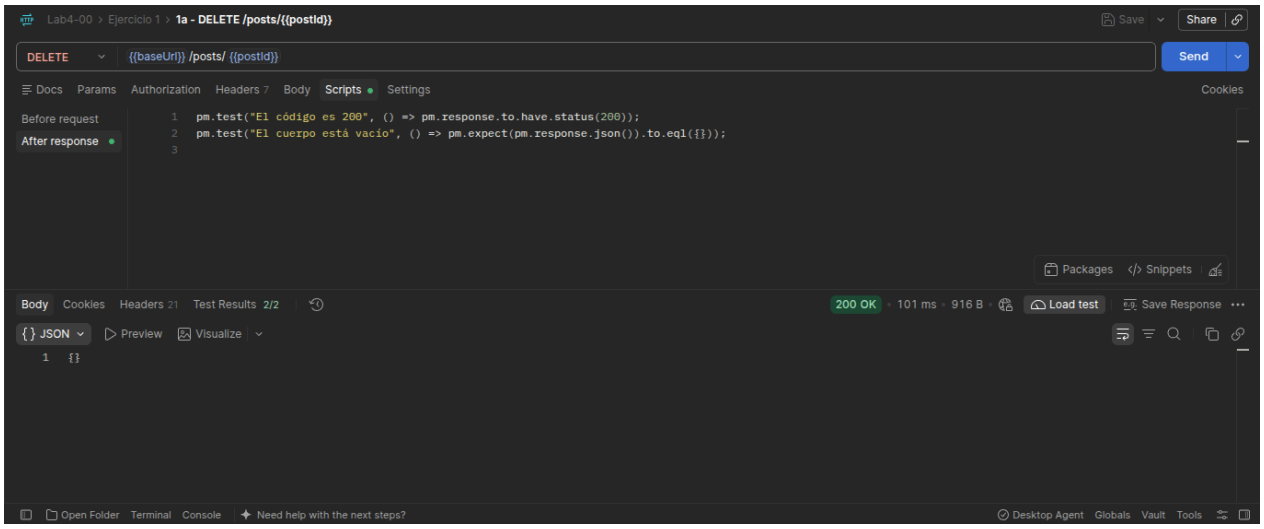


Figura 17. `DELETE /posts/1`.

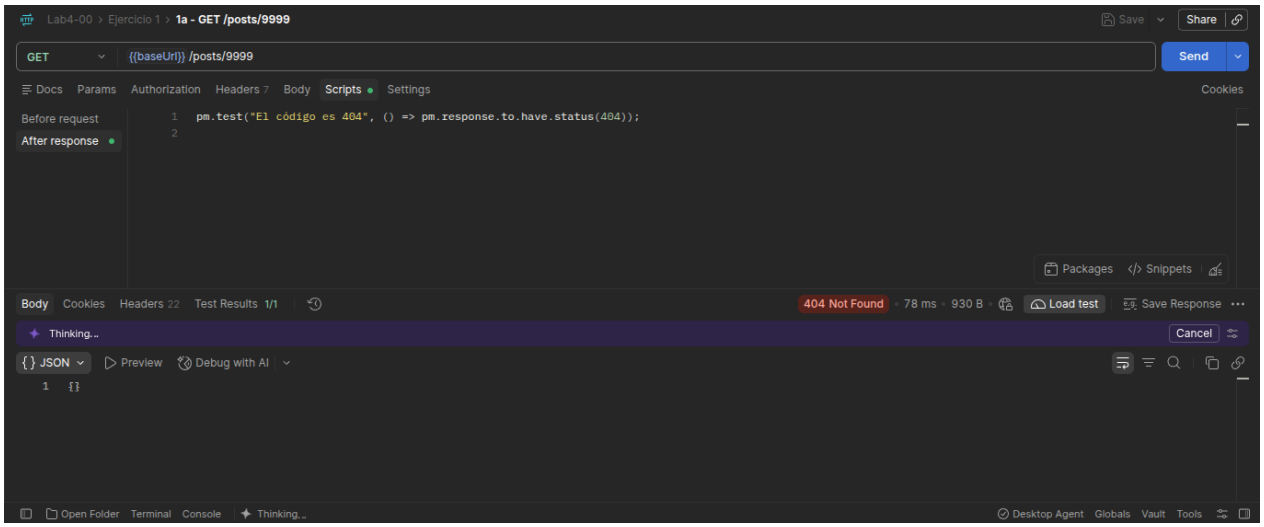


Figura 18. `GET /posts/9999`.

(b) Ruta anidada o parámetro de consulta

Las dos peticiones devuelven los mismos 10 posts, pero dicen cosas distintas. La ruta `/users/1/posts` dice que los posts *pertenecen* al usuario 1: es una relación de pertenencia. El parámetro `/posts?userId=1` dice que de la colección de todos los posts se quieren solo algunos: es un filtro.

El criterio general es que la ruta identifica un recurso y la consulta filtra una colección. Conviene la ruta anidada cuando el hijo no tiene sentido sin el padre y casi siempre se consulta desde él, como las líneas de un pedido (`/pedidos/7/lineas`); conviene un nivel de anidamiento, dos como máximo. Conviene el parámetro cuando el criterio es opcional o se combina con otros: `/posts?userId=1&q=lab&page=2` se arma sin crear rutas nuevas, mientras que con rutas habría que inventar una por cada combinación.

(c) Crear y luego consultar

El `POST /posts` respondió `201 Created`, un cuerpo con `id: 101` y el header `Location: https://jsonplaceholder.typicode.com/posts/101`. El `GET /posts/101` siguiente respondió `404 Not Found`.

JSONPlaceholder solo tiene 100 posts y no guarda las escrituras: calcula el `id` que tendría el post y arma la respuesta que daría un servidor real, pero no lo almacena. Por eso la URL del `Location` lleva a un recurso que no existe. Una prueba guarda el `id` en la variable `nuevoId`, y la petición siguiente consulta `{{baseUrl}}/posts/{{nuevoId}}`, sin escribir el 101 a mano.

En una API real, el `POST` debe responder `201 Created` con el header `Location` apuntando al recurso nuevo, y el `GET` a esa URL debe devolverlo con `200`. JSONPlaceholder imita bien la respuesta; lo que falta es el almacenamiento.

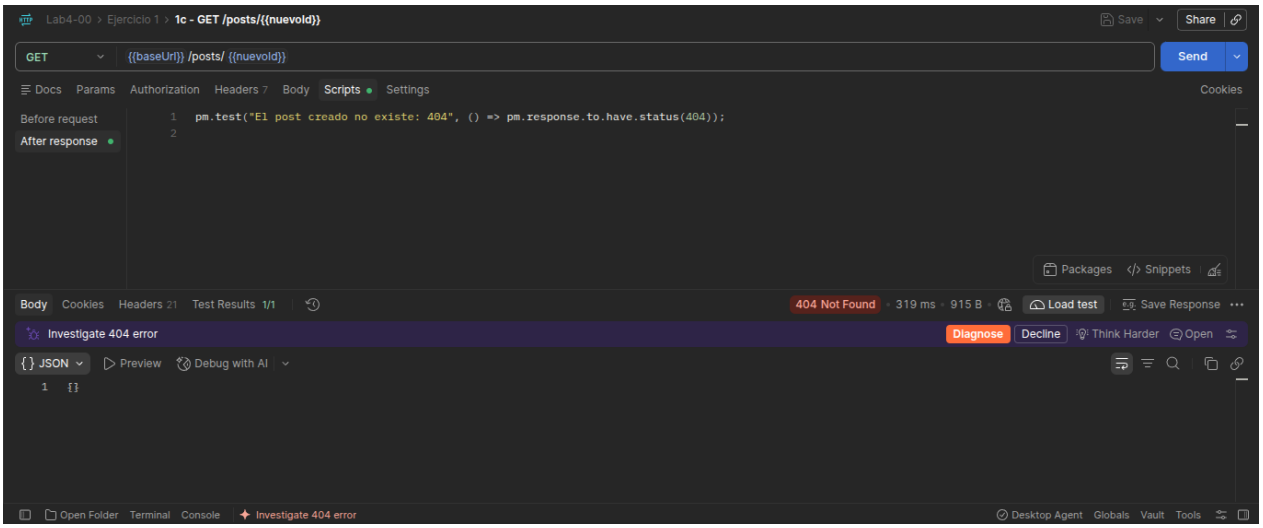


Figura 19. `GET /posts/{{nuevoId}}` después del `POST`: 404.

(d) PUT y PATCH

El `PUT /posts/1` envía el recurso completo:

```
{ "id": 1, "title": "Titulo nuevo", "body": "Cuerpo nuevo", "userId": 1 }
```

El `PATCH /posts/1` envía solo lo que cambia:

```
{ "title": "Solo cambia el titulo" }
```

La respuesta del `PATCH` trae el título nuevo y el `body` original del post 1 (`"quia et suscipit…"`), porque los campos que no se enviaron se conservan.

`PUT` significa "deja el recurso exactamente así". Si se omite `title`, el post queda sin título: JSONPlaceholder lo muestra, porque su respuesta al `PUT` sin `title` trae solo `id`, `body` y `userId`. En una API real pasaría una de dos cosas: si `title` es obligatorio, el servidor rechaza la petición con `400 Bad Request`; si es opcional, lo borra o lo deja en `null`. En ningún caso conserva el título anterior: para cambiar un solo campo se usa `PATCH`.

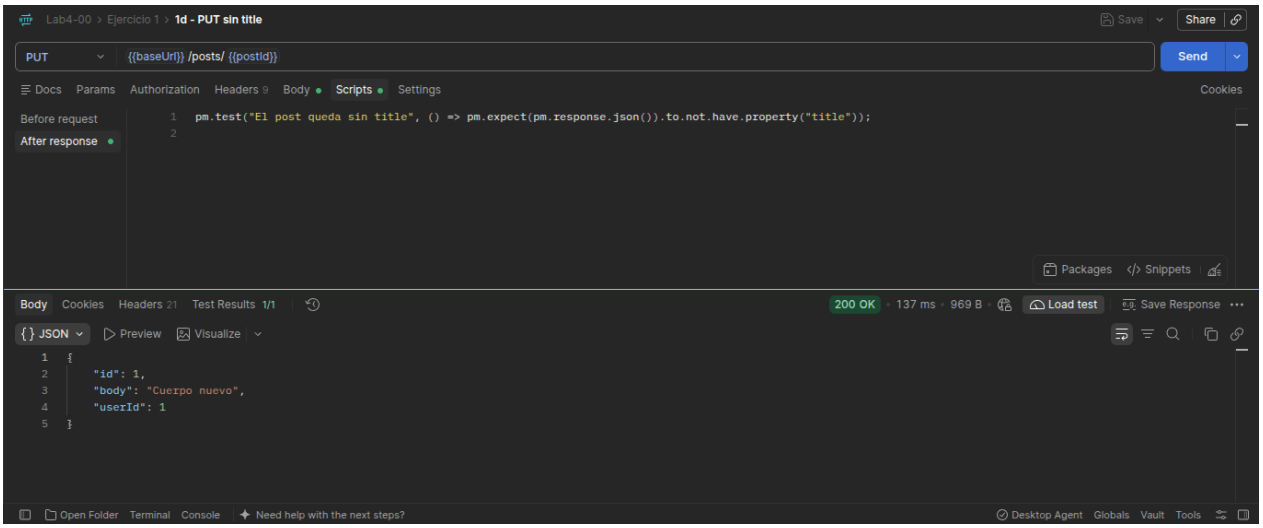


Figura 20. `PUT /posts/1` sin `title`: la respuesta no lo trae.

(e) Pruebas automáticas

Las diez peticiones de la carpeta `Ejercicio 1` tienen pruebas en *Scripts → Post-response*. Todas verifican el código de estado; además, cada una revisa lo propio de su método: que los filtros solo traigan posts del usuario 1, que el `POST` incluya `Location`, que el `PATCH` conserve el `body` y que el `PUT` sin `title` lo pierda. El archivo completo es `postman/scripts-ejercicio1.js`. Las de `GET /users` son estas:

```
pm.test("El código es 200", () => pm.response.to.have.status(200));
pm.test("Responde en menos de 1 s", () => pm.expect(pm.response.responseTime).to.be.
    below(1000));
pm.test("Devuelve una lista de 10 usuarios", () => {
    const usuarios = pm.response.json();
    pm.expect(usuarios).to.be.an("array").that.has.lengthOf(10);
    pm.expect(usuarios[0]).to.have.property("email");
});
```

El Collection Runner ejecutó la carpeta completa, en orden, con el environment `Lab4-00`: las 19 pruebas pasaron, sin errores, en 2,8 segundos. El orden importa: el `POST` va antes del `GET /posts/{{nuevoId}}` porque es el que define `nuevoId`.

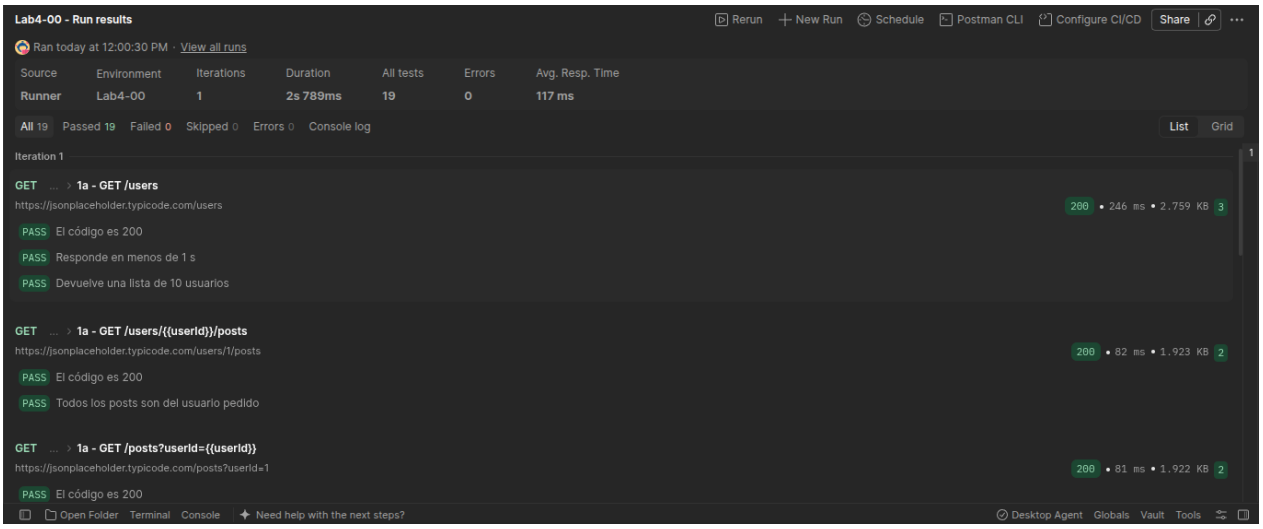


Figura 21. Resultado del Collection Runner sobre la carpeta `Ejercicio 1`.

Ejercicio 2. Un cliente REST en Java

El proyecto Maven está en la carpeta `cliente-java` del repositorio. Hace las peticiones con `java.net.http.HttpClient` y convierte el JSON con Gson 2.11.0. El paquete `dominio` tiene los registros `Clima` y `Pais`, las interfaces de los repositorios y las excepciones; `infraestructura` tiene las clases que hablan HTTP. El dominio no importa nada de `java.net` ni de Gson.

REST Countries ya no sirve. La URL de la guía, https://restcountries.com/v3.1/name/colombia?fields=name,capital,population,latlng, hoy responde `301 Moved Permanently` hacia otro servidor, y ese servidor responde `200` con `"success": false` y el mensaje *This API version has been deprecated*. Responde igual para Colombia y para Wakanda. La versión nueva, la v5, exige una llave: sin ella responde `401` con `authKeyMissing`, y con la llave de demostración devuelve siempre a Canadá. Por eso el programa usa `countries.dev`, un servicio gratuito y sin llave que conserva las rutas de REST Countries (`/name/{nombre}`, con `?fields=`). Lo que cambia es la forma del JSON: `name` y `capital` son texto en vez de objeto y lista. La clase se llama `CountriesDevPaisRepository`, y no `RestCountriesPaisRepository`, porque lleva el nombre de la API con la que habla.

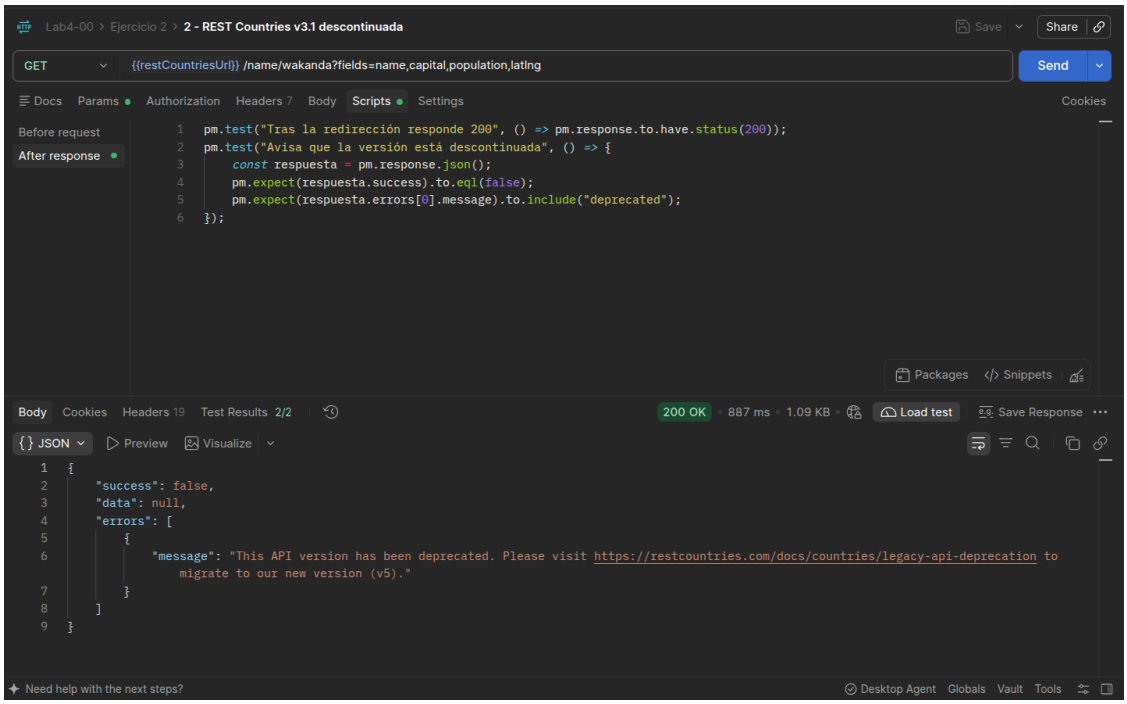


Figura 22. REST Countries v3.1 hoy: la versión está descontinuada.

(a) Los repositorios

En `OpenMeteoClimaRepository`, los cuatro TODO quedaron así:

```
try {
    HttpResponse<String> res = http.send(req, HttpResponse.BodyHandlers.ofString());
    if (res.statusCode() != 200) {
        throw new ClimaNoDisponible("Open-Meteo respondió " + res.statusCode() + motivo
    (res.body()));
    }
    JsonObject actual = JsonParser.parseString(res.body())
        .getAsJsonObject().getAsJsonObject("current");
    return new Clima(actual.get("temperature_2m").getAsDouble(),
                     actual.get("wind_speed_10m").getAsDouble());
} catch (HttpTimeoutException e) {
    throw new ClimaNoDisponible("Open-Meteo no respondió en 5 segundos", e);
} catch (IOException e) {
    throw new ClimaNoDisponible("No hubo conexión con Open-Meteo", e);
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();
    throw new ClimaNoDisponible("La consulta del clima se interrumpió", e);
} catch (JsonParseException | IllegalStateException
         | UnsupportedOperationException | NullPointerException e) {
    throw new ClimaNoDisponible("Open-Meteo respondió en un formato inesperado", e);
}
```

La conexión y la respuesta tienen cada una un límite de 5 segundos. `HttpTimeoutException` se atrapa antes que `IOException` porque es una subclase, y así el mensaje distingue “tardó demasiado” de “no hay red”. `InterruptedException` vuelve a marcar el hilo como interrumpido antes de traducirse, para que quien llamó se entere. El último `catch` cubre un cuerpo que no es JSON o al

que le falta `current`. Cuando Open-Meteo responde `400`, explica el motivo en el campo `reason`, y el mensaje lo incluye.

`PaisRepository` declara dos excepciones distintas: `PaisNoEncontrado` (el servicio respondió, pero el país no existe) y `PaisNoDisponible` (el servicio falló). La diferencia importa porque lo que hace el usuario es distinto en cada caso: en el primero corrige el nombre; en el segundo, reintenta más tarde.

```
public interface PaisRepository {
    Pais buscar(String nombre) throws PaisNoEncontrado, PaisNoDisponible;
}
```

`CountriesDevPaisRepository` lee el JSON con Gson en un registro privado que copia la forma de la respuesta, y después lo convierte en `Pais`. Así, la forma del JSON de countries.dev no sale de esa clase. Tres detalles: el nombre se codifica en la URL (`Costa Rica` viaja como `Costa%20Rica`); la búsqueda es por coincidencia parcial y `india` trae también el Territorio Británico del Océano Índico, así que gana el país cuyo nombre coincide exactamente; y la Antártida no tiene capital, así que `capital` puede llegar en `null`.

(b) El programa

`App.main` recibe el nombre del país en los argumentos. Se ejecuta con:

```
cd cliente-java
mvn -q compile exec:java -Dexec.args="Colombia"
```

Resultados del 5 de noviembre de 2026:

```
$ Colombia
Colombia (capital Bogotá, 50.882.884 hab.): 30,0 °C, viento 1,2 km/h
$ Costa Rica
Costa Rica (capital San José, 5.094.114 hab.): 19,4 °C, viento 2,4 km/h
$ india
India (capital New Delhi, 1.380.004.385 hab.): 23,8 °C, viento 5,1 km/h
$ Antarctica
Antarctica (sin capital, 1.000 hab.): -34,0 °C, viento 18,9 km/h
```

Los números salen con el formato de Colombia (`Locale es-CO`): punto para los miles y coma para los decimales. Los 30 °C de Colombia no son los de Bogotá: `latlng` es el centro geográfico del país (`4, -72`, en los Llanos), no la capital. Con las coordenadas de Bogotá (`4.61, -74.08`), Open-Meteo daba 17,7 °C a la misma hora. La población es la de 2020, la que trae countries.dev; el ejemplo de la guía usaba un dato más reciente.

(c) Un país inexistente

REST Countries v3.1 respondía `404 Not Found` a `/name/wakanda`. Hoy responde lo mismo que a cualquier otro país: `301` y, al seguir la redirección, `200` con el aviso de versión descontinuada. countries.dev responde `404 Not Found`, con un cuerpo que no es JSON: `Content-Type: text/plain` y el texto `Country not found`. Por eso el repositorio revisa el código antes de leer el cuerpo: si intentara convertir ese texto con Gson, fallaría. En Postman, las pruebas de esta petición verifican las tres cosas: el `404`, el `Content-Type` y el texto. Están en `postman/scripts-ejercicio2.js`, junto con las de las otras peticiones de la carpeta.

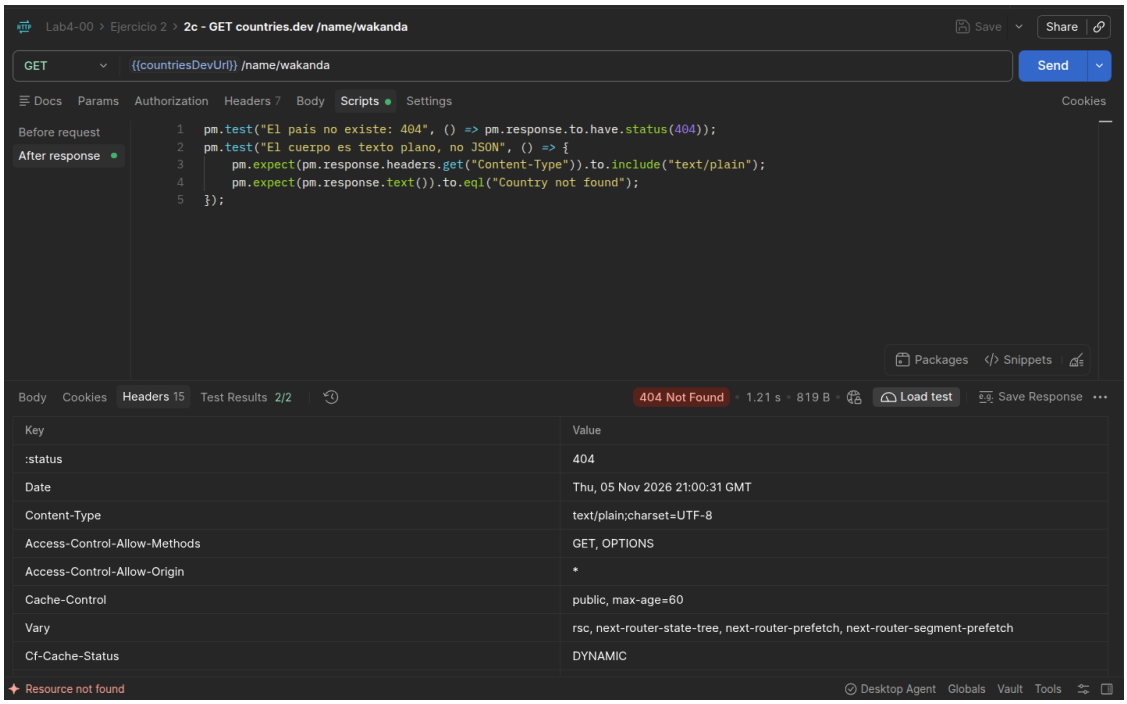


Figura 23. `GET https://countries.dev/name/wakanda`: 404 en texto plano.

El `404` se convierte en `PaisNoEncontrado`, y `main` la atrapa y muestra un mensaje. Las demás fallas también terminan en un mensaje: lo comprobé sin red y con un servidor que acepta la conexión pero nunca responde (el programa se rindió a los 5,8 segundos):

```
$ wakanda
No existe un país llamado "wakanda". Escriba el nombre en inglés, por ejemplo Spain o Germany.
$ Colombia        (sin red)
No se pudo completar la consulta: No hubo conexión con countries.dev. Intente más tarde.
$ Colombia        (servidor que no responde)
No se pudo completar la consulta: countries.dev no respondió en 5 segundos. Intente más tarde.
```

Los mensajes van por la salida de errores y el programa termina con código 1, para que un script que lo llame sepa que falló. Si no recibe un nombre, explica cómo usarlo y termina con código 2.

(d) Por qué `main` depende de la interfaz

El principio de inversión de dependencias dice que la lógica de alto nivel no debe depender de los detalles, sino de abstracciones, y que la abstracción la define quien la usa. Aquí, `ClimaRepository` vive en el paquete `dominio` y dice lo que el dominio necesita ("el clima en estas coordenadas"), sin mencionar HTTP, URLs ni JSON. `OpenMeteoClimaRepository` vive en `infraestructura` e implementa esa interfaz. La dependencia apunta de la infraestructura hacia el dominio, no al revés.

El cálculo del resumen está en el método `resumen`, que recibe un `PaisRepository` y un `ClimaRepository`. `main` es el único lugar que conoce las clases concretas: las crea en dos líneas y se las pasa. Este ejercicio lo puso a prueba sin buscarlo: cuando REST Countries dejó de servir, el cambio fue escribir `CountriesDevPaisRepository` y cambiar una línea de `main`. `resumen`, `Pais` y la interfaz no se tocaron. La misma separación permite probar `resumen` con repositorios falsos que devuelven datos fijos, sin red.

En DDD, un Repository es la abstracción con la que el dominio obtiene sus objetos como si estuvieran en una colección, sin saber de dónde salen. Su interfaz pertenece al dominio y su

implementación a la infraestructura. Aquí los datos no salen de una base de datos sino de una API, pero el papel es el mismo. Además, cada repositorio traduce el modelo ajeno al propio: el JSON de countries.dev (`name`, `latlng` como lista) se convierte en `Pais` (`nombre`, `lat`, `lon`) y los errores de red en excepciones del dominio. Así, los cambios de una API externa se quedan en una sola clase.

(e) En una app Android

El hilo. Android no deja hacer peticiones de red en el hilo principal, que es el que dibuja la pantalla: lanza `NetworkOnMainThreadException`. La petición tiene que correr en un hilo de fondo, por ejemplo con una corrutina en `viewModelScope` y `Dispatchers.IO`. El resultado se publica de vuelta para que la vista se actualice en el hilo principal. `viewModelScope` cancela la petición si el usuario cierra la pantalla.

El permiso. En el `AndroidManifest.xml` hace falta `<uses-permission android:name="android.permission.INTERNET" />`. Es un permiso normal: se concede al instalar, sin preguntarle al usuario. Las dos APIs son HTTPS, y eso importa, porque desde Android 9 el tráfico HTTP sin cifrar está bloqueado por defecto.

El cliente HTTP. Android no incluye `java.net.http.HttpClient`. Habría que cambiarlo por OkHttp o Retrofit, y ese cambio también queda dentro de las dos clases de infraestructura.

En MVVM, cada clase iría en una capa:

Capa	Qué hay
Model: dominio	`Clima`, `Pais`, `ClimaRepository`, `PaisRepository` y las excepciones. No cambian.
Model: datos	`OpenMeteoClimaRepository` y `CountriesDevPaisRepository`, reescritas con Retrofit u OkHttp.
ViewModel	Un `ClimaPaisViewModel` que recibe los repositorios por constructor, hace lo que hoy hace `resumen` y expone el estado de la pantalla (cargando, resultado o mensaje de error) como `StateFlow` o `LiveData`. Las excepciones se convierten aquí en mensajes, como hoy en `main`.
View	La `Activity`, el `Fragment` o la pantalla de Compose: un campo para el país y un botón. Observa el estado y lo dibuja; no hace peticiones.

`main` desaparece. La creación de los repositorios pasa a una `ViewModelProvider.Factory` o a un inyector de dependencias como Hilt.

Ejercicio 3. Los seis protocolos en la práctica

Las peticiones HTTP están en la carpeta `Ejercicio 3` de la colección `Lab4-00`, con pruebas en *Scripts* → *Post-response*. Las pruebas están en `postman/scripts-ejercicio3.js`. La conexión WebSocket está en otra colección, `Lab4-00 WebSocket`, porque Postman guarda las peticiones WebSocket en un tipo de colección distinto al de las HTTP.

(a) REST y GraphQL

La URL REST de la guía (https://restcountries.com/v3.1/alpha/co) tiene el mismo problema del ejercicio 2: responde con el aviso de versión descontinuada. La petición REST va a

countries.dev, que conserva la misma ruta: `GET {{countriesDevUrl}}/alpha/co`, sin `fields`. La GraphQL es un `POST` a `https://countries.trevorblades.com/graphql` con la consulta de la guía en el cuerpo; Postman la envía como `{"query": "..."}`.

Petición	Cuerpo de la respuesta	Total en Postman (con headers)	Qué trae
REST	689 bytes	1,24 KB	24 campos: área, coeficiente de Gini, bandera, fronteras, husos horarios, códigos telefónicos, dominio de internet…
GraphQL	110 bytes	954 B	Los 4 campos pedidos: `name`, `capital`, `currency` y `languages`.

El cuerpo REST es seis veces más grande, y de sus 24 campos la pantalla solo necesitaba 3. Eso es el *overfetching*: el servidor decide la forma de la respuesta y el cliente recibe todo, lo use o no. En GraphQL decide el cliente: la consulta nombra los campos y la respuesta trae solo esos. Una de las pruebas lo verifica en cada ejecución: compara el tamaño de las dos respuestas y exige que la GraphQL pese menos. Los totales de Postman están más cerca entre sí (1,24 KB contra 954 B) porque la respuesta GraphQL trae 23 headers y la REST 15: en una respuesta tan pequeña, los headers pesan tanto como los datos.

En una red móvil esto importa por cuatro razones. Cada byte cuenta contra el plan de datos del usuario. La radio del teléfono consume batería mientras transmite. La latencia es alta y variable, y una respuesta más grande tarda más en llegar completa. Y el teléfono tiene que convertir todo el JSON aunque descarte casi todo. Con un país la diferencia es de 579 bytes; en una lista de 250 países, o en una pantalla que se recarga muchas veces, el desperdicio se multiplica. REST también puede evitarlo, si la API lo ofrece: `?fields=name,capital,currencies` en countries.dev, como en el ejercicio 2. La diferencia es que en REST lo decide quien diseña cada endpoint, y en GraphQL viene incluido en el lenguaje de consulta.

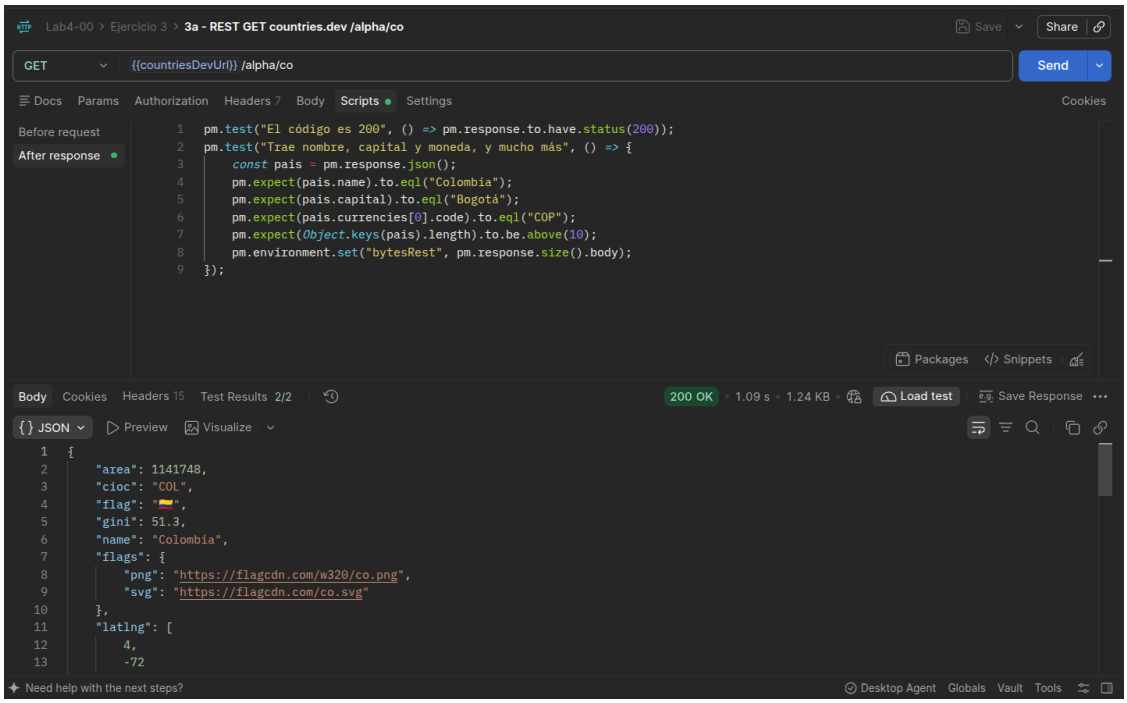


Figura 24. REST: `GET /alpha/co` sin `fields`, 1,24 KB.

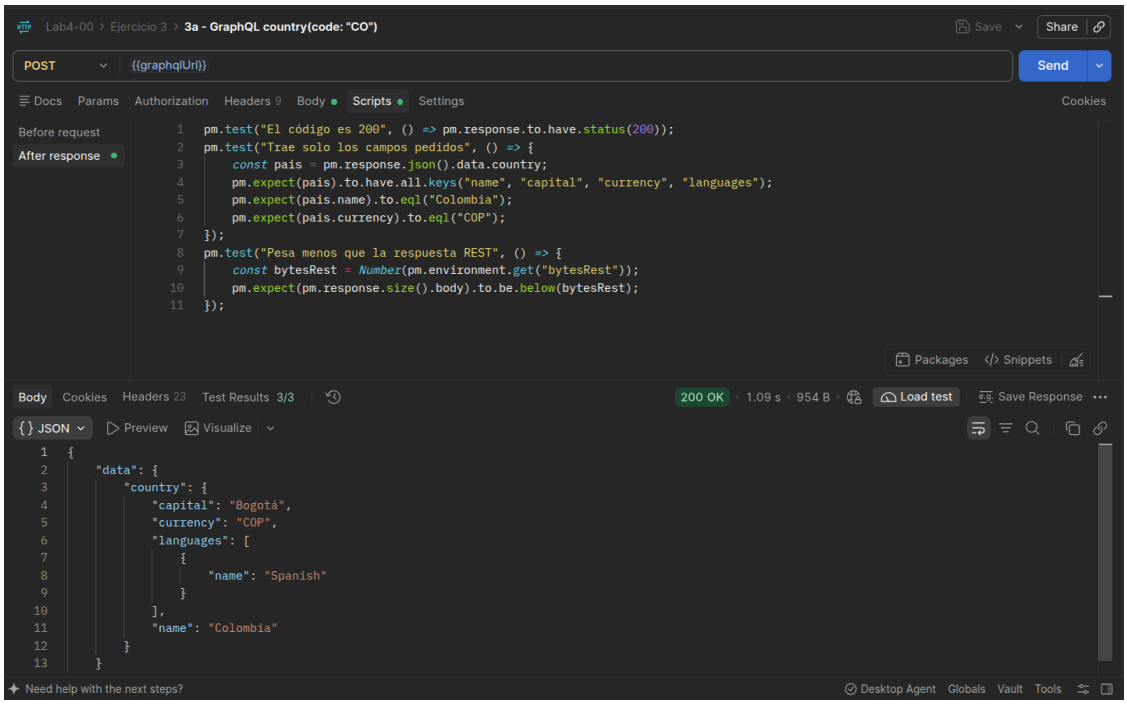


Figura 25. GraphQL: la misma información en 954 B.

(b) SOAP

El servicio de la guía, `www.dataaccess.com`, corta la conexión desde este equipo: falla por HTTPS y por HTTP, y también su página principal. Lo reemplacé por el *CountryInfoService* de `webservices.oorsprong.org`, que es de la misma empresa (Data Access) y corre en la misma plataforma (DataFlex), así que el sobre y la respuesta tienen la misma forma. Con la operación `FullCountryInfo` y el código `CO` pide los datos de Colombia, los mismos del literal (a), así que el ejercicio queda con tres maneras de pedir la misma información. Como no hay número que convertir, el número del grupo no se usa en este literal.

La petición es `POST` a http://webservices.oorsprong.org/websamples.countryinfo/CountryInfoService.wso con el header `Content-Type: text/xml; charset=utf-8` y este sobre:

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <FullCountryInfo xmlns="http://www.oorsprong.org/websamples.countryinfo">
      <sCountryISOCode>CO</sCountryISOCode>
    </FullCountryInfo>
  </soap:Body>
</soap:Envelope>
```

En la respuesta (figura 26):

Envelope. `<soap:Envelope>` envuelve todo el mensaje y declara con `xmlns:soap` que es SOAP 1.1. Aquí no trae `<soap:Header>`, que es opcional; ahí iría, por ejemplo, la firma de WS-Security.

Body. `<soap:Body>` contiene la respuesta de la operación, `<m:FullCountryInfoResponse>`. El nombre se arma con el de la operación más `Response`.

Resultado. `<m:FullCountryInfoResult>` trae los datos: `sName` `Colombia`, `sCapitalCity` `Bogota`, `sCurrencyISOCode` `COP` y, dentro de `Languages`, `Spanish`.

En SOAP, la operación no se elige con el método ni con la ruta, sino con el elemento que va dentro del Body: todas las llamadas son `POST` a la misma URL. El mismo contenido que GraphQL entregó en 110 bytes ocupa aquí 906 bytes sin comprimir, por las etiquetas de apertura y cierre y los espacios de nombres. El servicio solo funciona por HTTP: el sobre viaja sin cifrar, y cualquiera en la red puede leerlo o modificarlo.

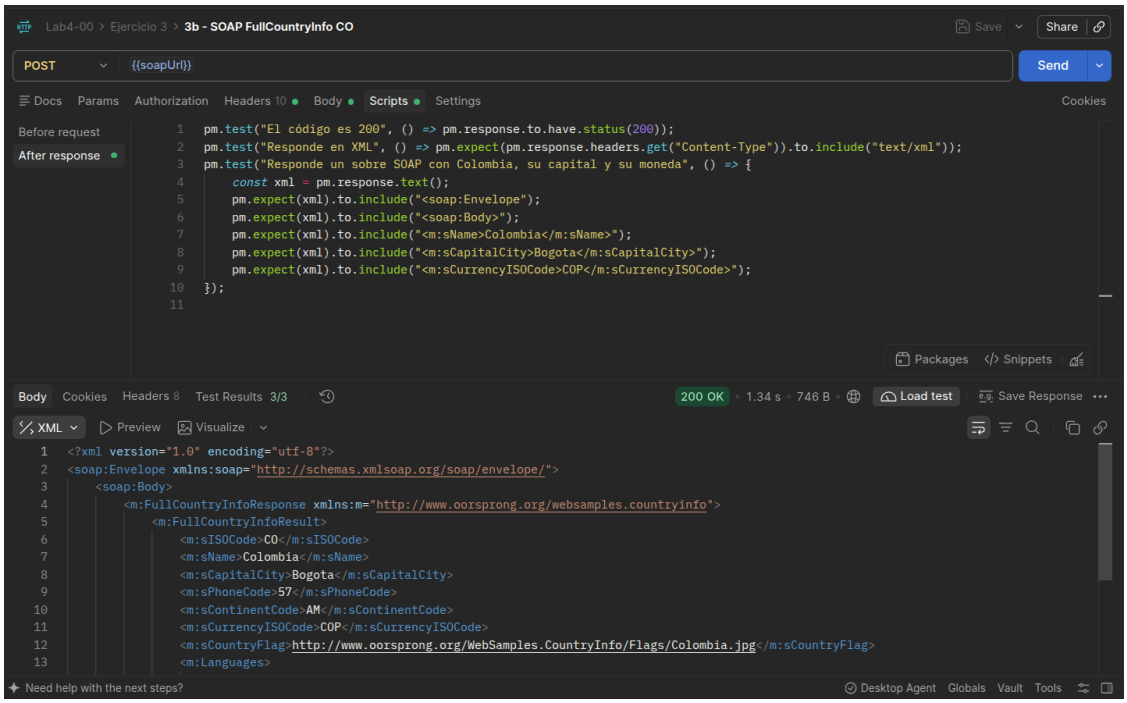


Figura 26. `FullCountryInfo` de Colombia: Envelope, Body y resultado.

El contrato de un servicio SOAP se publica como un documento WSDL (*Web Services Description Language*), en la misma URL del servicio con `?WSDL` al final. Es un XML que describe los tipos de datos (`types`), los mensajes de entrada y salida (`message`), las operaciones (`portType`, con `FullCountryInfo` y otras 20), cómo se transportan (`binding`) y la dirección del servicio (`service`, con `soap:address`). Con el WSDL, herramientas como `wsimport` en Java generan el cliente completo. Es el equivalente de la especificación OpenAPI del ejercicio 4, pero obligatorio y escrito para máquinas.

Este servidor responde distinto según cómo termine la URL. Con `?WSDL` exacto entrega el WSDL. Sin parámetro, o con `?WSDL=`, entrega una página HTML que lista las operaciones para leerlas en el navegador. Eso causó un problema: enviada sola, la petición recibía el WSDL, pero en el Collection Runner recibía el HTML y la prueba fallaba. El Runner mostraba que había enviado `?WSDL=`. La causa está en cómo Postman guarda la petición: el parámetro `WSDL` queda con valor vacío (`""`), y su librería lo convierte en `?WSDL=`; solo un parámetro sin valor (`null`) sale como `?WSDL`. La solución fue poner la URL completa en una variable del environment, `soapWsdlUrl`: así la petición guardada no tiene parámetros, y la URL se arma recién al enviarla, ya con `?WSDL` sin igual.

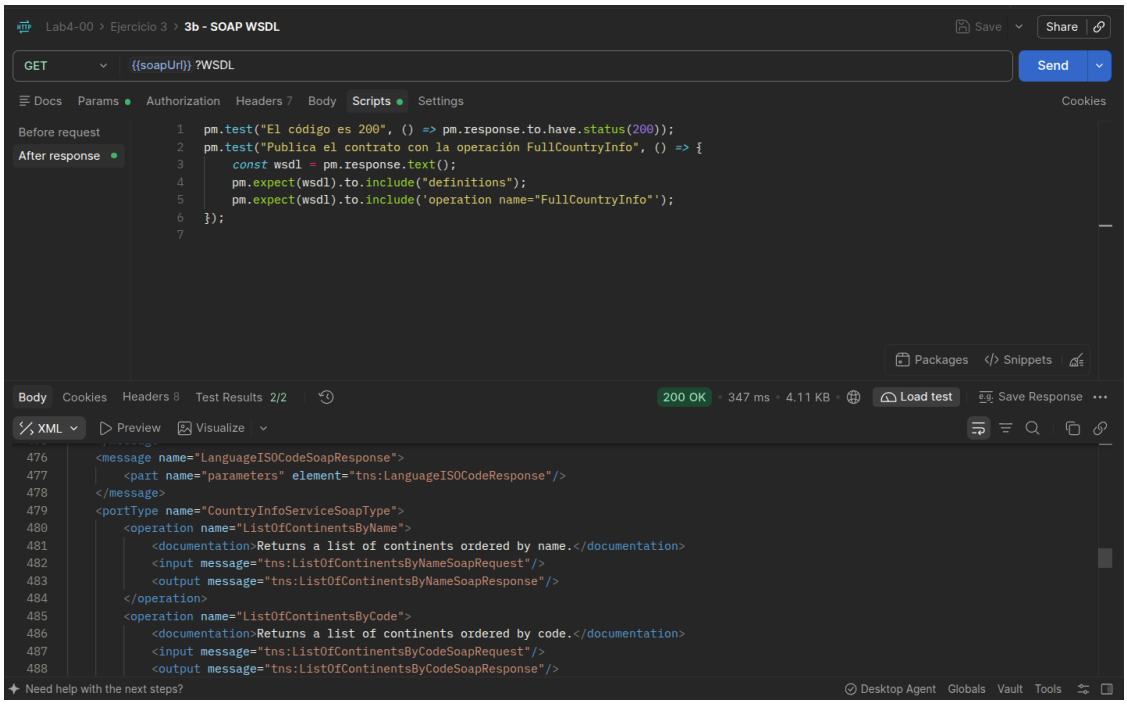


Figura 27. El WSDL del servicio: `portType` con sus operaciones.

(c) WebSocket

La conexión a `wss://ws.postman-echo.com/raw` se abrió a las 17:08:59, envió `hola`, `Musubi Japan` y `grupo 0`, recibió cada uno de vuelta entre 308 y 386 ms después, y se cerró a las 17:09:22.

La diferencia con las peticiones anteriores es que no hay peticiones: hay una conexión. En HTTP, cada intercambio es una petición del cliente seguida de una respuesta del servidor, y el servidor no puede hablar si no le preguntan. WebSocket empieza con una petición HTTP con el header `Upgrade: websocket`, el servidor responde `101 Switching Protocols`, y desde ahí la conexión deja de ser HTTP: queda abierta, y cualquiera de los dos lados envía mensajes cuando quiera, sin esperar al otro y sin repetir headers. Los mensajes no tienen método, URL ni código de estado; son texto (o binario) dentro de un marco de unos pocos bytes. Aquí el servidor solo devuelve el eco, pero podría enviar mensajes sin que nadie se los pida.

Para los tres mensajes se abrió **una sola conexión**: el panel muestra un solo *Connected* al principio y un solo *Disconnected* al final. Con HTTP habrían sido tres peticiones, cada una con sus headers completos.

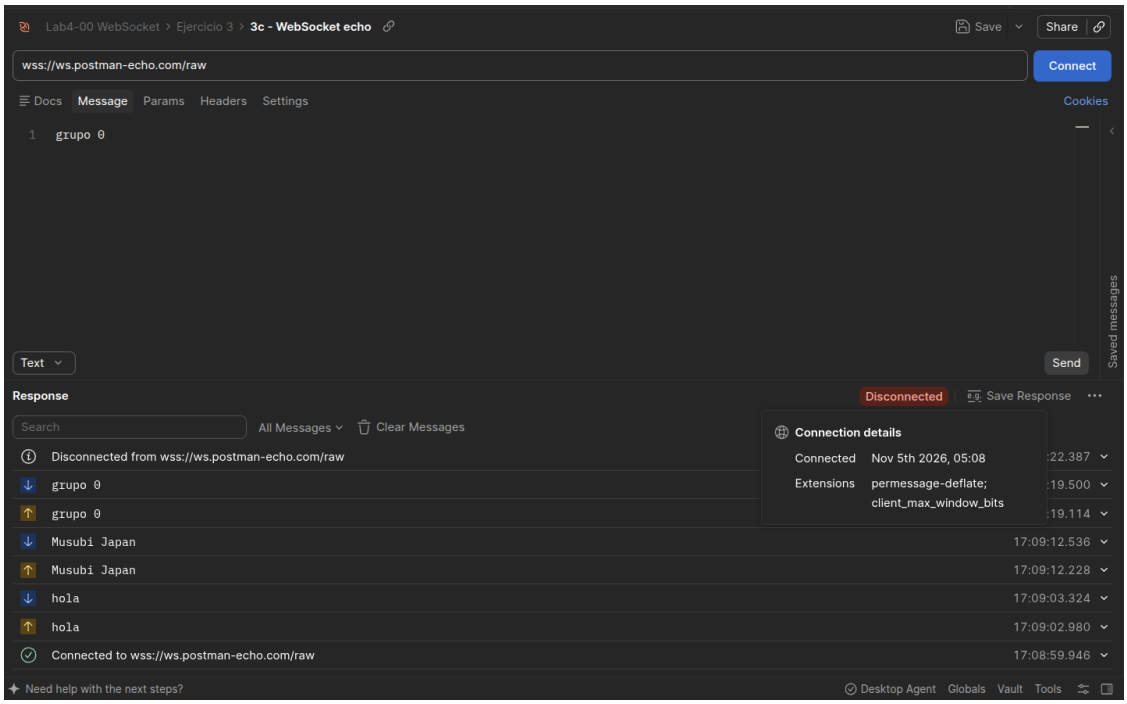


Figura 28. Una conexión, tres mensajes enviados y tres ecos.

(d) Webhooks

La petición fue un `POST` a la URL única de webhook.site con `{"evento": "pago.aprobado", "pedidoId": 123}`, y la página la mostró al instante con su método, sus headers y el cuerpo. En la captura difuminé la IP y la ubicación del equipo, que webhook.site muestra.

En un webhook la comunicación la inicia **quien produce el evento**: la pasarela de pagos llama a la aplicación cuando el pago se aprueba, en vez de que la aplicación le pregunte cada tanto si ya se aprobó. Los papeles se invierten: el sistema que recibe el aviso es el servidor, y tiene que exponer una URL pública para que lo encuentren.

Esa URL pública es el problema, y este ejercicio lo muestra: el aviso de "pago aprobado" que recibió webhook.site no lo envió ninguna pasarela, sino Postman (`user-agent: PostmanRuntime/2.14.0`). Cualquiera que conozca la URL puede hacer lo mismo. Antes de confiar en el mensaje, el receptor debe validar:

La firma. La pasarela firma el cuerpo con un secreto compartido (normalmente un HMAC-SHA256) y envía la firma en un header. El receptor recalcula la firma con su copia del secreto y la compara; si no coincide, descarta el mensaje. Así sabe quién lo envió y que nadie lo cambió en el camino.

La fecha. La firma debe incluir una marca de tiempo, y el receptor debe rechazar mensajes de hace más de unos minutos. Si no, alguien que capture un aviso válido puede reenviarlo después.

Que no esté repetido. Las pasarelas reintentan cuando no reciben un `2xx`, así que el mismo evento puede llegar varias veces. El receptor guarda el identificador de cada evento y no aprueba dos veces el mismo pedido.

Que los datos tengan sentido. Que el pedido 123 exista, esté pendiente y el monto coincida. Para un pago, lo más seguro es tomar el aviso solo como una señal y consultar el estado real en la API de la pasarela antes de entregar algo.

Además, la URL debe ser HTTPS, y el receptor debe responder `200` rápido y procesar el evento después, para que la pasarela no crea que falló y lo reintente.

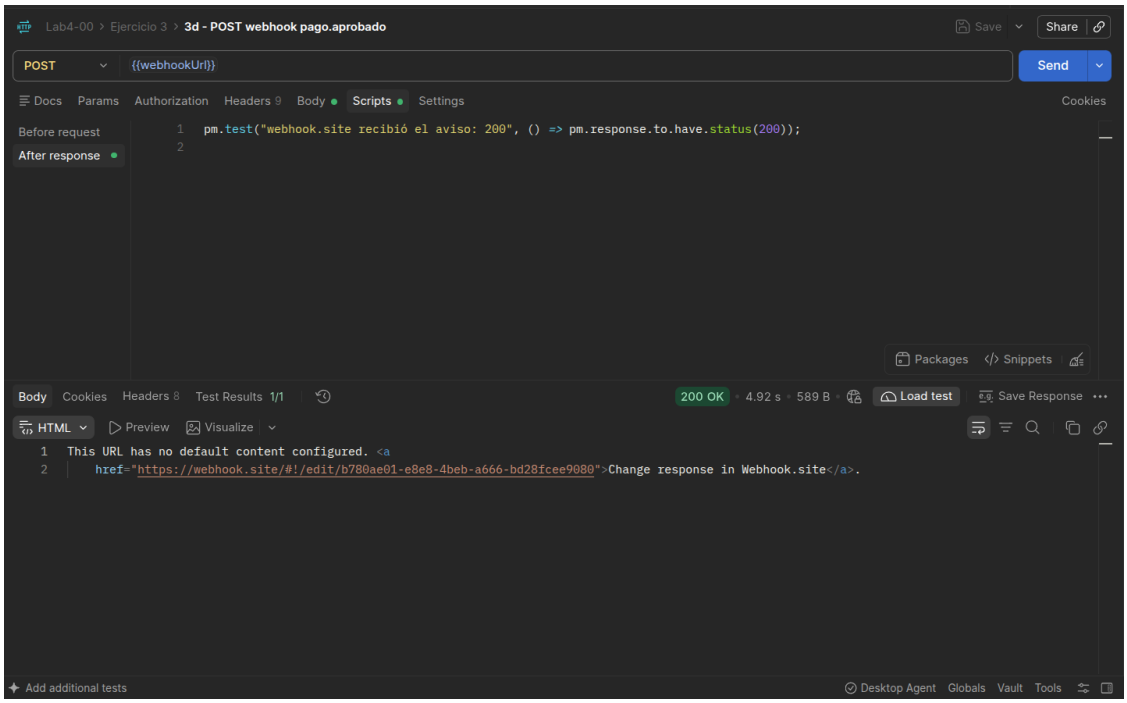


Figura 29. El aviso enviado desde Postman.

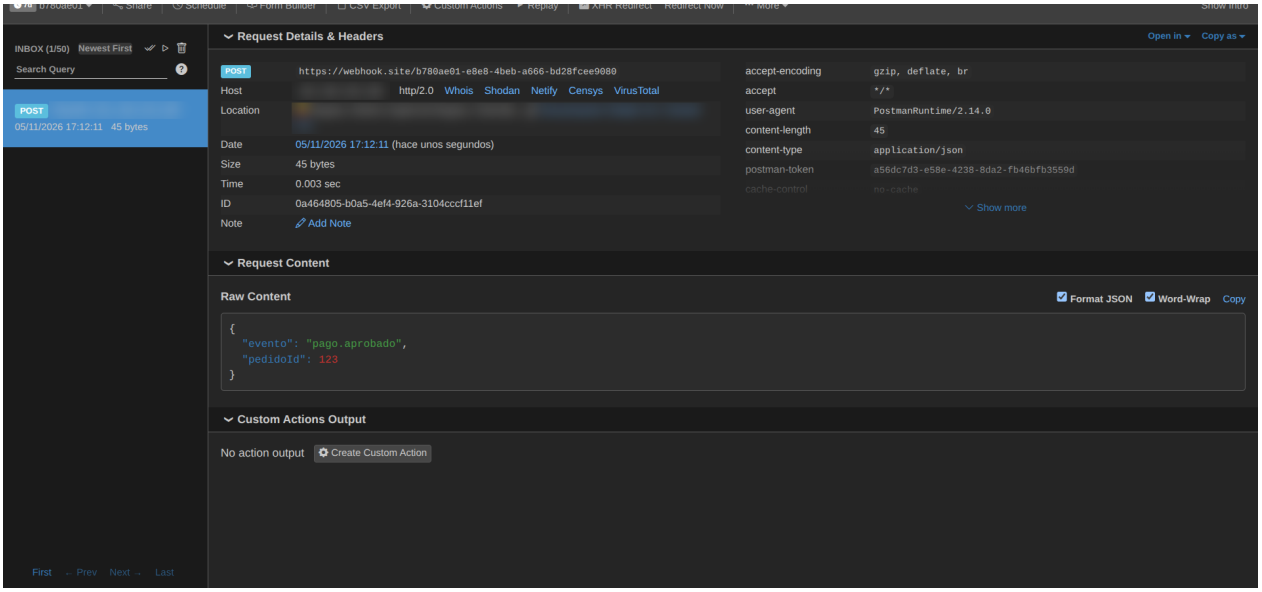


Figura 30. El aviso recibido en webhook.site (IP y ubicación difuminadas).

(e) gRPC

Un archivo `.proto` es el contrato de un servicio gRPC, escrito en el lenguaje de Protocol Buffers. Define los mensajes, con cada campo tipado y numerado, y los servicios con sus métodos. Por ejemplo, para la mascota de Musubi:

```
syntax = "proto3";

message Frase {
  string texto = 1;
  string nivel = 2;
}

message Correccion {
  bool correcta = 1;
  string sugerencia = 2;
  repeated string explicaciones = 3;
}

service Mascota {
  rpc Corregir (Frase) returns (Correccion);
  rpc Conversar (stream Frase) returns (stream Correccion);
}
```

Con el compilador `protoc` se genera, a partir de ese archivo, el código del cliente y del servidor en Java, Kotlin, Go, Python y otros lenguajes. El cliente llama a `Corregir` como si fuera un método local. Los mensajes viajan en binario: cada campo se identifica por su número (`1`, `2`, `3`), no por su nombre, y por eso pesan mucho menos que el JSON equivalente. Los números también permiten cambiar el contrato sin romper a los clientes viejos, siempre que no se reutilicen.

gRPC se usa más entre servicios de backend por tres razones:

El navegador no puede hablarlo. gRPC necesita HTTP/2 con control de sus marcos y de los *trailers*, donde viaja el estado de la llamada. El navegador usa HTTP/2 por dentro, pero no le da ese control a JavaScript. Para usarlo desde una página hace falta gRPC-Web y un proxy intermedio, como Envoy, que traduzca.

Entre servicios es donde más rinde. Los servicios de un backend se llaman miles de veces por segundo. Ahí importan los mensajes binarios, la conexión HTTP/2 que lleva muchas llamadas a la vez y el *streaming* en ambos sentidos, como en `Conversar`.

El contrato se comparte. Cliente y servidor deben compilarse con el mismo `.proto`. Eso es fácil cuando ambos los mantiene el mismo equipo, y difícil con clientes externos que no controlamos. Además, el binario no se lee ni se prueba a mano como un JSON, y para una API pública eso es una desventaja.

(f) Tabla comparativa

Los casos de uso son de Musubi Japan, la aplicación móvil para aprender japonés del proyecto de aula. El estudio funciona sin conexión; la red se usa para descargar contenido, para la mascota (una capa con IA que propone ejercicios y corrige frases) y para la suscripción que la paga.

Protocolo	Formato de datos	¿Quién inicia?	¿Conexión persistente?	Caso de uso en Musubi
REST	JSON sobre HTTP	El cliente	No: cada petición es independiente	Descargar paquetes de frases, kanji y vocabulario para estudiar sin conexión: `GET /paquetes/{id}`.
GraphQL	Consulta GraphQL; respuesta JSON	El cliente	No (salvo las *subscriptions*, que van por WebSocket)	La pantalla de una frase pide sus piezas con solo los campos que muestra: lectura, significado y nivel de cada kanji.
SOAP	XML en un sobre, con contrato WSDL	El cliente	No	Emitir la factura electrónica de cada pago de la suscripción ante la DIAN, cuyos servicios web son SOAP.
WebSocket	Mensajes de texto o binarios, casi siempre JSON	El cliente abre; después, cualquiera de los dos	Sí	La práctica de conversación con la mascota: la corrección llega mientras se genera, y la mascota puede intervenir sin que se le pida.
Webhook	JSON en un `POST` HTTP	El proveedor del evento	No: un aviso por evento	La pasarela de pagos o Google Play avisa al servidor que la suscripción se aprobó, se renovó o se canceló.
gRPC	Protocol Buffers (binario) sobre HTTP/2	El cliente; con *streaming*, los dos envían	Sí: una conexión HTTP/2 con muchas llamadas	Entre el servidor de Musubi y el servicio de IA de la mascota, que se llaman muchas veces y comparten el `.proto`.

Ejercicio 4. Diseño y documentación de una API REST

La guía permite usar el dominio del proyecto de aula en lugar del préstamo de equipos, y esta API usa el de Musubi, la aplicación móvil para aprender japonés. Musubi enseña con frases, y cada frase se desglosa en *piezas* de cuatro tipos: kana, kanji, vocabulario y gramática. El estudio funciona en el teléfono sin conexión, y el avance del aprendiz nunca sale de ahí.

La API es la de **contenidos**: el catálogo de piezas, frases, lecturas y la ruta sugerida. La aplicación trae el kana y los primeros temas de la ruta, y descarga el resto cuando tiene conexión: la ruta, el diccionario y los trazos por paquetes, y las lecturas una por una. Así no depende de traer todo el contenido ni satura el teléfono, y el contenido crece sin publicar una versión nueva de la aplicación. El equipo de Musubi escribe los contenidos por la misma API y los publica por versiones. Hasta ahora, las decisiones del proyecto decían que los contenidos venían completos dentro de la aplicación; este diseño queda como propuesta para cambiarlas.

La API tiene dos clientes con permisos distintos. La aplicación solo lee, de forma anónima: leer contenido no necesita saber quién es el aprendiz. El equipo escribe con un token que tiene rol de editor.

(a) Recursos y endpoints

Los recursos son seis:

Piezas. Kana, kanji, vocabulario y gramática. Cada una tiene un identificador definitivo, tomado de la fuente cuando existe: el punto de código Unicode para kana y kanji (`kanji-5b66` es

学), el número de entrada del diccionario JMdict para el vocabulario (`vocab-1206900` es 学生) y uno propio para la gramática.

Frases. Enunciados que atan piezas, como わたしは学生です。 (“Soy estudiante”).

Lecturas. Textos para practicar. Se descargan una por una.

Ruta. El grupo de frases que ofrece la aplicación, ordenado por temas.

Paquetes. Lo que la aplicación descarga de una vez: un tema de la ruta, un bloque del diccionario o los trazos.

Versiones. Cada publicación del contenido, con lo que cambió.

La API tiene 19 operaciones en 12 rutas, todas bajo `https://api.musubi.app/api/v1`. Las de escritura exigen el token de editor y pueden responder además `401` (sin token) y `403` (sin rol de editor). Las de lectura pueden responder `429` si una IP supera el límite de peticiones.

Método	Ruta	Descripción	Cuerpo	Éxito	Errores
GET	`/paquetes`	Manifiesto: qué paquetes hay, de qué versión y cuánto pesan	—	200	400
GET	`/paquetes/{id}`	Descarga un paquete	—	200, 304	404
GET	`/versiones?desde=41`	Lo que cambió después de una versión	—	200	400
POST	`/versiones`	Publica los borradores como una versión	`notas`	201	409: no hay cambios
GET	`/versiones/{numero}`	Una versión publicada	—	200	404
GET	`/piezas`	Busca y filtra; con `forma`, completa las palabras de una frase propia	—	200	400
POST	`/piezas`	Crea una pieza en borrador	`id`, `tipo`, `forma` y los campos de su tipo	201	400; 409: identificador usado o pieza duplicada
GET	`/piezas/{id}`	Detalle, también de una pieza retirada o fusionada	—	200	404
PATCH	`/piezas/{id}`	Corrige, retira o fusiona una pieza	`tipo` y los campos que cambian	200	400, 404; 409: ya fusionada, en uso o reemplazo no publicado
GET	`/piezas/{id}/frases`	Frases en las que aparece	—	200	404
GET	`/ruta/temas`	Los temas de la ruta, en orden	—	200	—
POST	`/frases`	Crea una frase en borrador	`texto`, `traduccion`, `tema`, `piezas`	201	400; 409: ya está en el tema
GET	`/frases/{id}`	Una frase con sus piezas	—	200	404

Método	Ruta	Descripción	Cuerpo	Éxito	Errores
PATCH	`/frases/{id}`	Corrige o retira una frase	Los campos que cambian	200	400, 404; 409: ya está en el tema nuevo
DELETE	`/frases/{id}`	Borra una frase en borrador	—	204	404; 409: ya publicada
GET	`/lecturas`	Catálogo de lecturas, sin el texto	—	200	400
POST	`/lecturas`	Crea una lectura en borrador	`titulo`, `nivel`, `texto`, `piezas`	201	400
GET	`/lecturas/{id}`	Descarga una lectura	—	200	404
PATCH	`/lecturas/{id}`	Corrige o retira una lectura	Los campos que cambian	200	400, 404

(b) Reglas de diseño

Sustantivos en plural. Cada colección es un sustantivo en plural (`piezas`, `frases`, `lecturas`, `paquetes`, `versiones`), y un elemento se nombra con su identificador debajo: `/piezas/kanji-5b66`. Así, la URL de un recurso se deduce sin documentación.

Sin verbos en la ruta. Lo que hace la operación lo dice el método, no la URL. Retirar y fusionar una pieza son cambios de estado (`PATCH`), y publicar es crear una versión (`POST /versiones`). No hay rutas como `/publicar` ni `/piezas/{id}/retirar`.

Jerarquía. `/piezas/{id}/frases` lista las frases de una pieza, que es lo que muestra la pantalla de detalle de Musubi, y `/ruta/temas` cuelga de la ruta. La jerarquía llega hasta un nivel: una frase se consulta en `/frases/{id}`, no en `/ruta/temas/7/frases/{id}`, porque su identificador ya es único.

Filtros y paginación. Los criterios opcionales van en la consulta y se combinan sin crear rutas: `/piezas?tipo=kanji&nivel=N5&q=estudio&page=1&size=20`. Las listas responden `{items, page, size, total}`, y `size` va de 1 a 100. Con `forma=食べる,学生`, la aplicación completa en una sola petición las palabras de una frase que escribió el aprendiz. Las envía sin repetir y en orden alfabético: las palabras sueltas no dicen nada del aprendiz, pero el orden de la frase sí.

Versionado. La ruta empieza por `/api/v1`. Esa es la versión del *contrato* de la API, y solo cambia si un cambio rompe a los clientes. Es distinta de la versión del *contenido* (42, 43…), que sube con cada publicación del equipo y viaja en los datos, no en la URL.

Los códigos de estado:

Código	Cuándo
200	Una lectura, o un `PATCH` que devuelve el recurso cambiado.
201	Se creó un recurso: una pieza, una frase, una lectura o una versión. Va con el header `Location`, que apunta al recurso nuevo.
204	Se borró una frase en borrador. No hay nada que devolver.
304	El paquete no cambió desde la copia de la aplicación (`If-None-Match`); no se descarga otra vez.
400	El cuerpo o la consulta no cumplen el formato: un campo que no corresponde al tipo de la pieza, `size=500`, una fusión entre piezas de tipos distintos.
404	El recurso no existe.
409	La petición tiene el formato correcto, pero choca con el estado del contenido: el identificador ya se usó, la pieza ya existe, la frase ya está publicada.
401, 403	Falta el token de editor, o el token no tiene ese rol.
429	Una IP superó el límite de peticiones; el header `Retry-After` dice cuándo reintentar.

La diferencia entre `400` y `409` es la que más se presta a confusión. El `400` se decide mirando solo la petición; el `409`, mirando lo que ya hay en el servidor. Crear la pieza `kanji-5b66` es una petición válida, y solo choca porque ese identificador ya existe.

(c) Una acción sin verbo en la URL

Musubi no presta equipos, así que no hay un préstamo que renovar. La acción equivalente es **fusionar una pieza duplicada**. Por regla, cada contenido tiene un solo código, y los duplicados se previenen al crear: `POST /piezas` responde `409` si ya hay una pieza activa del mismo tipo con la misma forma y las mismas lecturas. Aun así, puede colarse uno. Por ejemplo, el equipo escribe a mano la palabra りんご (`vocab-p-0007`), y después importa la entrada de JMdict de 林檎 (`vocab-1555480`), que es la misma palabra con sus kanji. Si hay aprendices con progreso en las dos, borrar la sobrante les haría perder ese progreso, y retirarla les dejaría la palabra como nueva. La pieza sobrante tiene que fusionarse en la otra.

Alternativas:

1. **Un cambio de estado.** `PATCH /piezas/vocab-p-0007` con `{"tipo": "vocabulario", "estado": "fusionada", "reemplazadaPor": "vocab-1555480"}`. La fusión es un estado de la pieza sobrante.

2. **Un recurso nuevo.** `POST /fusiones` con `{"origen": "vocab-p-0007", "destino": "vocab-1555480", "motivo": "…"}`. La fusión es un objeto con su propio identificador, que se puede consultar en `/fusiones/{id}` y cancelar con `DELETE` antes de publicarla.

Elegí la **primera**. La segunda conviene cuando la acción tiene vida propia: se consulta, se cancela, tiene su historia. Así sería un préstamo, que se solicita, se aprueba, se renueva y se devuelve. Aquí, los duplicados son raros, porque se previenen al crear, y no justifican mantener un recurso con sus operaciones. Lo que la aplicación necesita saber de una fusión ya viaja en la versión que la publica.

El mismo criterio da el resultado contrario al publicar. Publicar se modela como un recurso, `POST /versiones`, y no como un cambio de estado de cada borrador, porque una versión sí tiene vida propia: tiene número, la aplicación la consulta (`GET /versiones?desde=41`) y lista lo que cambió.

La fusión, además, tiene que ser determinista. El progreso está en cada teléfono y no en el servidor, así que el servidor no puede moverlo: solo puede avisar, y cada teléfono aplica el aviso por su cuenta. Para que todos terminen igual:

1. Solo se fusiona una pieza publicada en otra publicada del mismo tipo.
2. La fusión se aplica al publicar la versión, y es irreversible: el progreso combinado ya no se puede separar.
3. `GET /versiones` lista las fusiones de cada versión en orden, y la aplicación aplica las versiones en orden. Un teléfono que viene de la versión 30 y otro que viene de la 41 llegan al mismo estado.
4. `reemplazadaPor` siempre apunta a la última pieza de la cadena: si A se fusionó en B y después B en C, el de A es C.
5. La aplicación combina los dos progresos con una regla fija: en cada grado queda el más alto, el próximo repaso queda en la fecha más cercana y los errores recurrentes se juntan. Aplicar dos veces la misma fusión no cambia nada, porque la pieza sobrante ya no tiene progreso.

(d) Formato único de error

Todos los errores usan el formato Problem Details (RFC 9457). Su tipo de contenido es `application/problem+json`, distinto del `application/json` de las respuestas exitosas, así que el cliente sabe que recibió un error antes de leer el cuerpo. El equivalente de "el equipo ya está prestado" es fusionar una pieza que ya se fusionó:

```
HTTP/1.1 409 Conflict
Content-Type: application/problem+json

{
  "type": "https://api.musubi.app/problemas/pieza-fusionada",
  "title": "La pieza ya fue fusionada",
  "status": 409,
  "detail": "vocab-p-0007 ya se fusionó en vocab-1555480 en la versión 42. Una fusión
    no se deshace.",
  "instance": "/api/v1/piezas/vocab-p-0007"
}
```

`type.` Identifica el tipo de problema con una URI. El cliente decide qué hacer según `type`, no según el texto, que puede cambiar o traducirse.

`title.` Resumen del tipo de problema: es el mismo para todos sus casos.

`status.` El código HTTP, repetido en el cuerpo para quien solo guarde el cuerpo, como un registro de errores.

`detail.` Lo que pasó en este caso: qué pieza, en qué versión.

`instance.` La ruta de la petición que falló.

La especificación define once tipos de problema, cada uno con su ejemplo. Seis son conflictos (`409`): `identificador-usado`, `pieza-duplicada`, `pieza-fusionada`, `pieza-en-uso`, `destino-inactivo` y `frase-publicada`; los demás cubren `400`, `401`, `403`, `404` y `429`. El mismo formato sirve para todos, así que la aplicación los maneja con un solo código.

(e) Documentación con OpenAPI

La especificación está en `openapi.yaml`, en la raíz del repositorio, en OpenAPI 3.0.3. Swagger Editor la carga sin errores, y lo comprobé también con Redocly CLI, que revisa además las buenas prácticas, y con swagger-parser, que resuelve todas las referencias. Ninguno da errores ni advertencias. Contiene:

- Las 19 operaciones, con sus parámetros, cuerpos y todas sus respuestas.
- 31 esquemas en `components/schemas`. `Pieza` es un `oneOf` con discriminador por `tipo`: cada tipo (Kana, Kanji, Vocabulario, Gramatica) tiene sus campos, y los cambios de un `PATCH` se validan igual. Los campos que asigna el servidor, como `estado` y `version`, son `readOnly`, así que no aparecen en los cuerpos de las peticiones.
- Ejemplos en todas las respuestas, con datos reales: los identificadores de vocabulario son entradas de JMdict, verificadas en el diccionario.
- La seguridad: lectura pública (`security: []`) y escritura con un token *bearer* de editor.
- Una descripción general con las reglas que no caben en un esquema: los dos clientes, las dos versiones, un solo código por contenido y las reglas de los duplicados.

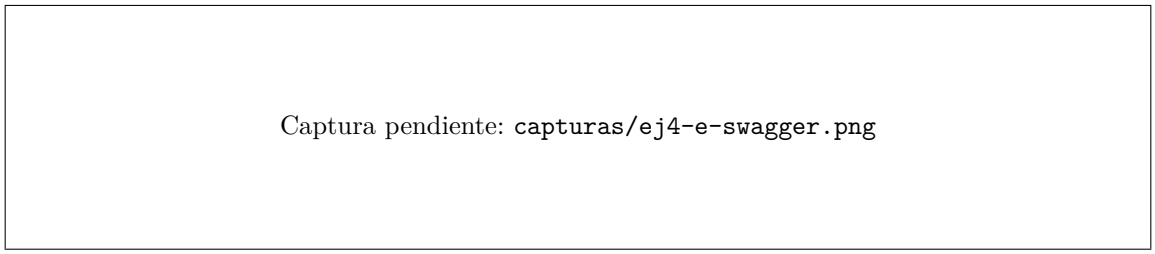


Figura 31. La documentación generada por Swagger Editor.

(f) Mock Server

Con un mock, la aplicación Android puede programar la descarga de paquetes, la búsqueda de palabras y el manejo de cada error antes de que exista el servidor: el mock responde con los ejemplos de la especificación. Para un equipo con frontend, móvil y backend en paralelo, la ventaja es que la especificación se vuelve el punto de acuerdo. Cada parte trabaja contra el contrato, no contra la implementación de otra, y nadie espera a nadie. Si algo del contrato no sirve, se descubre al principio, cuando cambiar el YAML cuesta poco, y no al integrar, cuando ya hay código de los dos lados.

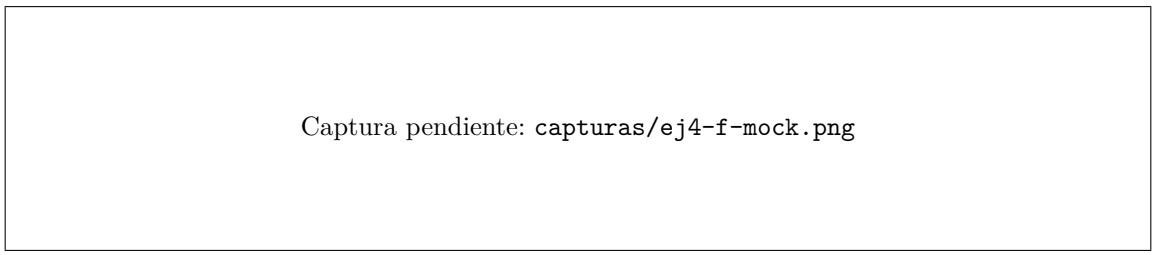


Figura 32. El Mock Server creado en Postman a partir de `openapi.yaml`.

Captura pendiente: `capturas/ej4-f-peticion.png`

Figura 33. Una petición contra el mock.